

CSGR: Compressed Source Gradient Replay for Scalable Self-Supervised Retriever Training

Anonymous ACL submission

Abstract

Dense retrievers are central to giving language models access to external knowledge, yet training them requires annotations largely absent in the domains—code, legal, medical, enterprise—where retrieval matters most. An annotation-free alternative lets the language model serve as supervisor: a candidate document is exposed to the model, the next-token prediction (NTP) loss is computed, and gradients are routed back through retriever scores. The obstacle is structural: NTP candidates are multi-layer KV memories the language model must read end-to-end, and unlike contrastive vectors they cannot be all-gathered across GPUs, pinning the candidate pool to a single device. We propose **Compressed Source Gradient Replay (CSGR)** to break this ceiling. **CSM** replaces raw-token source exposure with a small set of in-source learnable compression slots, cutting cross-document attention cost from quadratic in source length to linear in slot count while improving retrieval supervision quality. **SGR** all-gathers these compact memories into a distributed global pool and replays gradients through the source compressor with exact equivalence to full-batch backward. Together, CSGR scales NTP-supervised retriever training from a single-GPU pool of 8 candidates to a distributed pool of 256, matching or improving retrieval quality on CoIR.

1 Introduction

Dense retrievers (Karpukhin et al., 2020; Izacard et al., 2022a; Wang et al., 2022b) depend almost entirely on annotated query-document pairs for supervision, yet such annotations are largely absent in the domains where retrieval matters most—code, legal, medical, and enterprise corpora (Li et al., 2025). These domains lack relevance judgments to begin with, and mining hard negatives itself presupposes a strong retriever. An annotation-free alternative reuses what a pretrained language

model already has: if a candidate document is genuinely relevant to a target sequence, conditioning on it should lower next-token prediction (NTP) loss. This reduction in loss is an annotation-free relevance signal that can be backpropagated through routing weights to train the retriever without any query-document labels (Guu et al., 2020; Shi et al., 2023; Cai et al., 2026).

Scaling this direction, however, runs into a structural wall absent from contrastive training. In contrastive learning, a candidate is a single vector; vectors from all GPUs can be all-gathered at negligible cost and the negative pool grows linearly with device count. Under NTP supervision, a candidate is a stack of multi-layer KV caches that the language model must actually read while gradients propagate back through attention and routing. Any attempt to share it across devices either ships large activation buffers before every forward pass, distributes cross-attention with non-standard kernels, or detaches cached states—the last option severing source parameters from any NTP gradient. The candidate pool is consequently pinned to a single GPU’s memory; REVELA (Cai et al., 2026), the strongest realization of this paradigm, is confined to 8 candidates per step.

This bottleneck has two faces. **First: the full-token candidate is unnecessarily heavy.** The retriever update passes through a single document-level scalar regardless of how many source tokens the reader processes, so raw-token breadth is not required for quality supervision—a more compact source interface may be an equally good or better one. **Second: even a lighter candidate cannot be shared across devices without breaking gradient correctness.** Standard data parallelism assumes each sample’s forward-backward is self-contained; detaching at a device boundary stops gradients from flowing back to the source encoder.

We propose **Compressed Source Gradient Replay (CSGR)** to address both challenges. **CSM** re-

places raw-token source exposure with K in-source learnable compression slots that attend over the full document in a single self-attention pass and expose only their key-value states to the reader, substantially reducing per-layer cross-document cost while forming a cleaner supervision interface—our experiments show $K=8$ slots match or exceed full cross-document attention on CoIR. **SGR** all-gathers these compact memories into a distributed global pool, streams targets through it one at a time, and replays accumulated gradients through the source compressor in a single backward pass with exact gradient equivalence to full-batch computation. Together, CSGR scales NTP-supervised retriever training from a single-GPU cap of 8 candidates to a distributed pool of 256, matching or improving retrieval quality on CoIR (Li et al., 2025).

Contributions.

(1) We identify the scaling barrier that prevents NTP-supervised retriever training from using GPU clusters the way contrastive training does. A candidate is not a single vector but a multi-layer source memory whose gradients must remain connected to the source computation graph, so standard data parallelism cannot enlarge the pool without severing gradients to the source encoder; increasing the retrieval pool or model size therefore quickly turns into an activation-memory wall.

(2) We propose **CSGR**, which combines learnable source compression with global cache replay. **CSM** (Compressed Source Memory) replaces full-token source exposure with learned compression slots, reducing reader-side cross-document attention from full source-token KV access to a slot-sized interface; for comparable source and target lengths, this changes the cross-document term from quadratic in sequence length to linear in the number of slots. **SGR** (Source Gradient Replay) then all-gathers the compressed memories into a distributed global pool, streams NTP supervision over the pool, and replays the accumulated gradients through the source compressor in a single backward pass, recovering the full-batch gradient exactly; we provide both a proposition and numerical verification.

(3) Empirically, CSGR turns NTP-supervised retriever training from a single-GPU candidate-pool regime into a distributed global-pool regime. It expands the trainable pool from 8 to 256 documents ($32\times$, with pool size growing with the number of gathered devices under the same protocol). In the 3B setting, CSGR improves CoIR retrieval qual-

ity by +2.57 nDCG@10 over the strongest prior NTP-supervised baseline, showing that the added scale translates into retrieval quality rather than only system feasibility.

2 Related Work

NTP-supervised retrieval. Dense retrievers are trained with labels, mined negatives, synthetic queries, or teacher distillation (Karpukhin et al., 2020; Izacard et al., 2022a; Wang et al., 2022b; Xiao et al., 2023), and domain adaptation extends this to low-resource domains via generated queries or pseudo labels (Wang et al., 2022a; Bonifacio et al., 2022; Dai et al., 2023). A parallel line connects retrieved content to language-model likelihood: REALM trains a retriever end-to-end with masked LM (Guu et al., 2020), RAG and Atlas integrate retrieval into generation (Lewis et al., 2021; Izacard et al., 2022b), RETRO builds retrieval into the decoder architecture (Borgeaud et al., 2022), and reader-to-retriever distillation transfers reader evidence into retriever scores (Izacard and Grave, 2021). REPLUG scores candidates by perplexity reduction and ensembles their likelihoods; each candidate forward pass is independent, so its pool is bounded by sequential compute rather than the gradient-coupling obstacle we address (Shi et al., 2023). Revela is the direct predecessor: it trains a dense retriever via NTP loss with retriever-weighted cross-document attention, and its structural candidate-pool bottleneck—the inability to share multi-layer KV states across devices with gradient correctness—is the specific obstacle CSGR removes (Cai et al., 2026).

Scaling and compression: technical building blocks. For expanding the candidate pool in contrastive retrieval, in-batch negatives (Karpukhin et al., 2020), momentum queues (He et al., 2020), and GradCache (Gao et al., 2021) are the main tools. GradCache decouples the contrastive loss backward from the encoder backward, caching per-sample gradients and replaying a single encoder pass per document to recover exact full-batch gradients; the key enabler is that a contrastive candidate is a single vector. SGR generalises this to NTP-supervised training, where the candidate is a multi-layer KV memory and the replay must propagate through the CSM compressor across both source-memory and routing-score paths (Proposition 1). For source compression, learnable tokens prepended to aggregate context appear in prefix

tuning (Li and Liang, 2021), memory transformers (Burtsev et al., 2020), compressive transformers (Rae et al., 2019), Set Transformer (Lee et al., 2019), and Perceiver (Jaegle et al., 2021); these apply slot compression for inference efficiency or parameter reduction. CSM applies the same mechanism as a *supervision-interface design*: the compression point determines what evidence reaches the retriever’s score channel (Eq. 3), and our experiments show this choice shapes retrieval quality, not only memory cost.

3 Preliminaries

NTP-supervised retrieval. A dense retriever E_θ maps each sequence D_i to $r_i = E_\theta(D_i)$ and assigns target–source scores $s_{ij} = \text{sim}(r_i, r_j)$. For a top- K_s candidate set $\mathcal{S}_i$ obtained from a frozen retriever snapshot or a teacher index, scores are normalised into routing weights

$$\alpha_{ij} = \frac{\exp(s_{ij}/\tau)}{\sum_{k \in \mathcal{S}_i} \exp(s_{ik}/\tau)}, \quad j \in \mathcal{S}_i, \quad (1)$$

with temperature τ . A reader (a language model) predicts target tokens while conditioning on the target prefix and routed sources. For source-exposure pattern $\mathcal{A}$, the loss is

$$\mathcal{L}_{i,\mathcal{A}} = - \sum_t \log p_\phi(x_{i,t} \mid x_{i,<t}, \{D_j, \alpha_{ij}\}_{j \in \mathcal{S}_i}; \mathcal{A}). \quad (2)$$

The candidate selection itself is detached, so gradients reach the retriever only through $\partial \mathcal{L}_{i,\mathcal{A}} / \partial s_{ij}$ over the routed candidates; there is no qrel-style direct signal. Under this supervision scheme, the candidate’s representation at each routed layer ℓ is the key-value cache $(K_j^\ell, V_j^\ell) \in \mathbb{R}^{L \times 2Hd_k}$, which the reader must attend to in the forward pass while gradients propagate back through cross-attention to the source-side parameters.

The score channel. Equations 1–2 have an immediate consequence that motivates the rest of the paper. For any source-exposure pattern $\mathcal{A}$, the chain rule converts every reader-side interaction with candidate D_j —across all target tokens, all routed layers, all attention heads, and all token-level cues the reader exploits—into a single centred scalar that updates the retriever score s_{ij} :

$$-\frac{\partial \mathcal{L}_{i,\mathcal{A}}}{\partial s_{ij}} = \frac{\alpha_{ij}}{\tau} \left(U_{ij}^{\mathcal{A}} - \sum_{k \in \mathcal{S}_i} \alpha_{ik} U_{ik}^{\mathcal{A}} \right), \quad (3)$$

$$U_{ij}^{\mathcal{A}} = -\frac{\partial \mathcal{L}_{i,\mathcal{A}}}{\partial \alpha_{ij}}.$$

We give the derivation in Appendix D; for our purposes Eq. 3 is an accounting identity, not an optimisation result: it shows that the source-exposure pattern $\mathcal{A}$ shapes what evidence reaches the retriever, making it a supervision-quality knob as well as an efficiency one.

4 Method

Model Overview. CSGR comprises two components. **CSM** (Section 4.1) addresses Challenge 1 by replacing the full-token source interface with K in-source learnable compression slots. **SGR** (Section 4.2) addresses Challenge 2 by all-gathering the compact memories into a distributed global pool and replaying gradients through the source compressor with exact gradient equivalence. Figure 1 shows the full training pipeline.

4.1 Compressed Source Memory (CSM)

Challenges Analysis. In NTP-supervised retrieval, each candidate’s token-by-token KV states must remain resident throughout the forward pass so the reader can cross-attend to every source position, and its intermediate activations must be retained until the backward completes so NTP gradients can propagate back to the source encoder. The number of source computation graphs live at once therefore equals the candidate pool size, bounded by a single GPU’s memory. Growing the pool requires a proportionally larger activation budget, imposing a hard memory ceiling that gradient accumulation alone cannot relieve.

Method Rationale. To address this challenge, we propose to replace the full-token source interface with a compact, learned representation formed inside the source self-attention pass. The key insight is that, since all reader evidence is ultimately compressed into a single scalar before reaching the retriever, what matters is not the breadth of token surface exposed but the quality of the signal passing through the score channel. Towards this end, CSM appends K learnable compression slots to the source sequence, processes them jointly with source tokens in a single self-attention pass, and exposes only the resulting slot key-value states to the reader. Because the slots attend over the full document before any reader interaction, they form a contextualized bottleneck that can be trained end-to-end to retain retrieval-relevant content while suppressing token-surface noise—reducing both

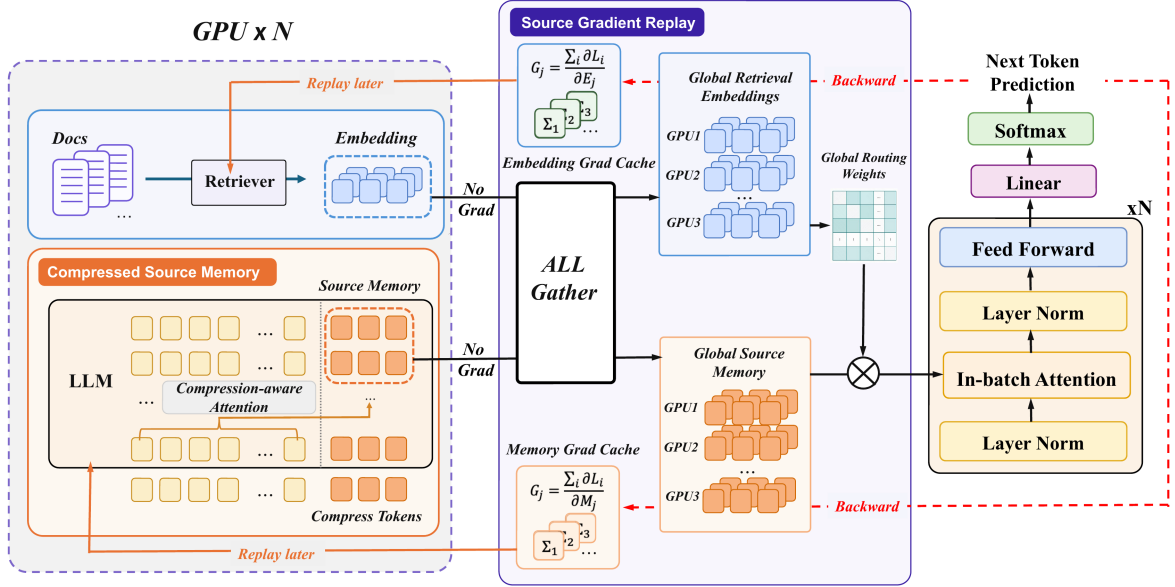


Figure 1: CSGR training pipeline. Each GPU encodes documents into retrieval embeddings and compressed source memories. The compact memories and embeddings are all-gathered into global pools, targets consume them through routed next-token prediction, and SGR replays accumulated memory and embedding gradients back through the source-side computation.

memory cost and score-channel distortion simultaneously.

Slot Compression. Let $D_j = (x_{j,1}, \dots, x_{j,L})$ be a source sequence of L tokens. CSM appends K learnable compression slots $c = (c_1, \dots, c_K)$, with $c_k \in \mathbb{R}^d$ shared across sources and trained end-to-end, to the source input and runs a single forward pass of the source encoder. After the pass, only the slot key-value states at each routed layer ℓ are retained:

$$\begin{aligned} M_j^\ell &= \text{slice}_{L+1:L+K} \text{KV}^\ell([D_j; c_1, \dots, c_K]), \\ M_j^\ell &\in \mathbb{R}^{K \times 2Hd_k}, \end{aligned} \quad (4)$$

where H is the number of attention heads and d_k the per-head key dimension. The reader at routed layer ℓ attends to M_j^ℓ (size K) instead of the raw source KV (size L), and the variable reader cost per target drops from $O(BK_s|\mathcal{R}|TL)$ to $O(BK_s|\mathcal{R}|TK)$, a factor of L/K over FULL-ATTN. Section 5.2 shows that $K=8$ matches or exceeds FULL-ATTN as a retriever supervision interface across CoIR tasks despite exposing far fewer states to the reader.

4.2 Source Gradient Replay (SGR)

Challenges Analysis. Even with compact slot memories, sharing them across devices while preserving gradient correctness runs into a wall that standard data parallelism cannot resolve. Any cross-device sharing attempt faces the same three dead ends: all-gathering the full KV states is communication-prohibitive and requires large activation buffers for the backward; distributing cross-attention requires non-standard kernels; and caching states as detached leaf tensors severs the computation graph from the source compressor, leaving it without any NTP gradient. Standard data parallelism assumes each sample’s forward-backward is self-contained—a condition violated here because the NTP loss of a target depends on the live computation graph of a source that may sit on another device.

Method Rationale. To address this, we observe that CSM introduces a clean architectural boundary: the slot states $M_j = \{M_j^\ell\}_{\ell \in \mathcal{R}}$ are the exclusive interface between the source encoder and the reader, and they constitute a natural detachment point that standard data parallelism can exploit. Towards this end, SGR encodes each source once, caches its slot memory as a detached leaf tensor, all-gathers

memories into a global pool across all devices, and streams targets through the pool one at a time. Because the source representation is fully decoupled at the slot boundary before being consumed by any target, the forward-backward of each sample becomes self-contained again; gradients that accumulate at the leaf tensors during streaming are then replayed through the source compressor in a single backward pass, recovering full-batch gradient equivalence without re-running the reader.

Encode–cache–stream–replay. The protocol runs in four stages.

(1) *Encode once.* Each source D_j passes through the CSM compressor once to produce M_j . The output is stored as a leaf tensor $\hat{M}_j \leftarrow \text{detach}(M_j)$, decoupled from the source encoder’s computation graph.

(2) *All-gather.* Leaf memories from all G devices are all-gathered into a global pool $\{\hat{M}_j\}_{j=1}^{GB_{\text{loc}}}$. Communication volume is $O(GB_{\text{loc}}|\mathcal{R}|Kd)$ —proportional to K , not L .

(3) *Stream targets.* Each target D_i computes its NTP loss $\mathcal{L}_i$ by attending to $\{\hat{M}_j\}_{j \in \mathcal{S}_i}$ one target at a time, so peak reader memory per step is $O(|\mathcal{R}|TK)$ regardless of pool size. Routing weights α_{ij} are recomputed live from the retriever scores, so gradients along the routing path (Eq. 3) flow without interruption. Each accumulated gradient $G_j \leftarrow \partial \mathcal{L}_i / \partial \hat{M}_j$ is added to a buffer attached to $\hat{M}_j$.

(4) *Replay.* After all targets are streamed, each $\hat{M}_j$ holds $G_j = \sum_{i: j \in \mathcal{S}_i} \partial \mathcal{L}_i / \partial \hat{M}_j$. A single backward re-runs the CSM compressor for D_j with G_j as the upstream gradient, propagating to all source encoder and slot parameters. Source encoding is therefore performed at most once per document per training step.

Gradient equivalence. Let θ_s denote the parameters of the CSM compressor.

Proposition 1 (Gradient equivalence of SGR). *Let $\mathcal{L} = \sum_i \mathcal{L}_i$ be the total NTP loss over a batch. Under the SGR protocol,*

$$\begin{aligned} \nabla_{\theta_s} \mathcal{L} \Big|_{\text{SGR}} &= \nabla_{\theta_s} \mathcal{L} \Big|_{\text{full-batch}}, \\ \frac{\partial \mathcal{L}}{\partial s_{ij}} \Big|_{\text{SGR}} &= \frac{\partial \mathcal{L}}{\partial s_{ij}} \Big|_{\text{full-batch}}. \end{aligned} \quad (5)$$

The proof follows from the chain rule applied to both the source-memory and routing-score gradient paths; see Appendix A.

A numerical verification (cosine similarity and relative ℓ_2 error between SGR and full-batch gradients across training steps) is reported in Appendix A.

Pool scalability. After SGR, the forward-backward of each target is self-contained: it reads detached cached memories and accumulates local gradients. Batches are therefore re-partitionable under standard data parallelism, and the effective pool size grows to $GB_{\text{loc}}K_s$ without any hardware-topology-specific kernel changes. Table 1 summarises the three configurations.

Table 1: Reader-side variable memory and candidate-pool scalability. B : per-device targets; K_s : candidates per target; $|\mathcal{R}|$: routed layers; T : target positions; G : devices.

Config.	Peak reader memory	Pool size	G-scale
FULL-ATTN	$O(BK_s \mathcal{R} TL)$	$B_{\text{loc}}K_s$	No
CSM only	$O(BK_s \mathcal{R} TK)$	$B_{\text{loc}}K_s$	No
CSM+SGR	$O(\mathcal{R} TK)$ per stream step	$GB_{\text{loc}}K_s$	Yes

5 Experiments

We evaluate CSGR from three complementary angles: whether the full system improves the quality–efficiency trade-off, whether CSM is a reliable source representation rather than an arbitrary compression trick, and whether SGR is gradient-correct while supporting larger candidate pools. Following this structure, we first describe the common setup and then report the main results, ablations, and system checks.

5.1 Experimental Setup

Benchmarks and metrics. CoIR is the main in-domain benchmark because code retrieval makes the weak-supervision problem concrete: identifiers, APIs, templates, and formatting can help token prediction without always indicating the desired retrieval relation (Li et al., 2025). We report nDCG@10 on the ten CoIR subtasks listed in Table 2. BEIR is kept as a cross-domain direction check: models are trained only on the code corpus and evaluated on the natural-language BEIR paper-13 suite without any BEIR-side fine-tuning (Thakur et al., 2021).

Training protocol. The matched comparison uses a Llama-3.2-1B backbone for both reader and

Table 2: Main CoIR task results (nDCG@10, %). External baselines are reproduced from CoIR and Revela (Li et al., 2025; Cai et al., 2026); † marks supervised or weakly supervised code-pair training and API columns are zero-shot. FULL-ATTN is the matched 1B local reference; CSGR rows are completed g256 checkpoint-5606 evaluations. Bold/underline mark the best/second-best score per row.

Task Size/status	BM25 —	E5-PT [†] 0.3B	BGE [†] 0.6B	REPLUG 1B	Revela 1B	FULL-ATTN 1B	CSGR 1B	Revela 3B	CSGR 3B	E5-Mis. [†] 7B	Ada-2 API	Voyage-2 API
Apps Retrieval	4.80	10.60	14.70	13.90	19.40	17.98	22.90	<u>26.60</u>	31.81	23.50	8.70	26.50
CosQA	15.60	27.10	26.40	20.10	30.20	31.90	32.55	29.00	33.47	<u>33.20</u>	28.90	29.80
Synthetic Text2SQL	25.00	48.90	46.90	53.90	55.00	57.03	59.36	55.90	61.90	<u>68.00</u>	58.30	69.30
CoIR CodeSearchNet	40.90	35.20	58.30	50.80	64.00	79.52	82.10	62.60	77.16	67.40	74.20	<u>81.80</u>
CodeSearchNet CC	54.00	50.50	53.70	62.80	70.00	74.66	<u>76.81</u>	69.10	77.13	64.80	53.30	<u>72.80</u>
CodeTransOcean Contest	47.80	56.30	62.60	61.50	81.10	74.83	76.19	83.20	<u>81.77</u>	80.60	26.00	27.30
CodeTransOcean DL	<u>34.40</u>	32.20	30.20	33.30	34.20	33.98	33.74	34.50	33.62	31.70	17.70	28.70
StackOverflow QA	70.20	86.90	80.60	76.30	85.70	85.52	89.96	88.30	91.25	<u>91.00</u>	74.20	81.80
CodeFeedback ST	68.10	70.40	69.30	66.00	76.20	75.95	77.73	<u>78.80</u>	79.04	76.40	47.10	65.40
CodeFeedback MT	59.20	46.20	47.90	42.90	<u>70.40</u>	56.57	59.16	73.00	59.60	36.40	17.70	28.70
Mean	42.00	46.40	49.10	48.20	58.60	58.79	<u>61.05</u>	60.10	62.67	57.30	45.60	56.30

retriever, trained on the Revela code training corpus for one epoch with LoRA (rank 256), DeepSpeed ZeRO-3, bf16, per-GPU batch size 4, 4 companion documents per target, gradient accumulation 4, and 8×80GB GPUs. All source-exposure rows share the same data, optimizer, training steps, routing temperature, NTP objective, and evaluation pipeline; they differ only in the source-exposure pattern. Three seeds (42–44) are run for the main endpoint comparison. For APPENDED LATENTS, $K=8$ is a default capacity fixed before the post-hoc capacity sweeps; no validation-set selection over K is performed. No CoIR qrel labels are used in training. Full hyperparameters and the command template are in Appendix G.

Baselines. Following the CoIR and Revela evaluations (Li et al., 2025; Cai et al., 2026), we compare against lexical retrieval (BM25), supervised or weakly supervised embedding models that used code-related query–document pairs during pre-training or fine-tuning (UniX, E5-PT, BGE, and E5-Mistral), zero-shot proprietary APIs (Ada-2 and Voyage-2), and NTP-style retriever-training baselines (REPLUG and Revela). Published external scores are copied from the source papers; we do not re-run those systems. The main task-level table reports the FULL-ATTN local reference and CSGR global-pool endpoints; the CSM local controls are kept in the matched ablation and appendix control tables.

Source-exposure patterns. We compare six source-exposure patterns that differ in compression locus and learnability: FULL-ATTN (full source-token KV), EOS STATE and MEAN STATE (non-

learned single-state summaries), APPENDED LATENTS (in-source learnable slots; the CSM interface, default $K=8$), EXTRACTED LATENTS (external cross-attention over frozen hidden states), and QUERY EVIDENCE (non-parametric token selection). Full definitions and readable-state counts are in Appendix C.

5.2 Results and Analysis

We evaluate CSGR across three angles: system-level quality and efficiency, the CSM source-interface design, and SGR gradient correctness and scalability.

Main CoIR results. Table 2 compares CSGR against all baselines on CoIR nDCG@10. (1) CSGR achieves the strongest mean among all compared systems, improving over the matched FULL-ATTN reference from 58.79 to 61.05 at 1B and reaching 62.67 at 3B. (2) CSGR outperforms supervised embedding models of comparable size on most tasks and matches or exceeds zero-shot proprietary APIs on the overall mean. These results demonstrate that NTP-supervised training with a distributed candidate pool is competitive with supervised retrieval without any query-document labels.

System ablation. Table 3 decomposes the system gain under matched 1B conditions, isolating the contributions of CSM, pool size, and SGR replay. (1) CSM local already improves over FULL-ATTN local (+1.17 nDCG@10) while reducing peak memory from 72.3 GB to 52.3 GB, confirming that compact in-source representation is a strictly better supervision interface. (2) The completed

Table 3: Matched 1B ablation on CoIR (nDCG@10, %). Pool is the number of candidate documents seen by each target; peak is rank-0 reserved GPU memory.

Method	Pool	CoIR	Peak
FULL-ATTN local	8	58.79	72.3GB
CSM local ($K=8$)	8	59.96	52.3GB
CSGR ($K=8$)	256	61.05	52.95GB

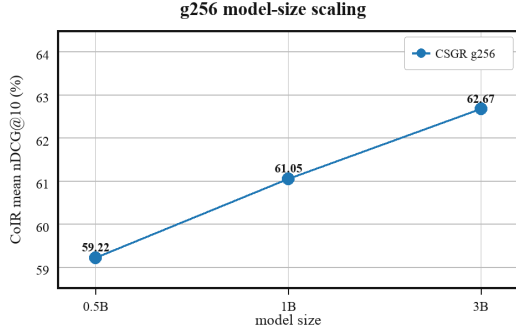


Figure 2: g256 model-size scaling on full CoIR evaluations. All rows use checkpoint-5606 evaluations with 10/10 CoIR tasks completed.

CSGR endpoint reaches 61.05 with a 256-doc global pool; the stop-gradient cache and full-token global rows remain the matched attribution controls.

Figure 2 confirms the gain holds consistently across 0.5B, 1B, and 3B model scales under the g256 setting. Together, CSM improves the supervision interface and SGR makes the distributed candidate pool gradient-correct.

CSM as a supervision interface. We compare six source-exposure patterns to verify that CSM is not merely a cheaper substitute for full-token attention. (1) APPENDED LATENTS $K=8$ outperforms FULL-ATTN, all single-state summaries, externally compressed latents, and query-selected evidence at both 1B and 3B scale (Figure 4; Appendix Table 11), demonstrating that in-source slot compression provides a superior supervision interface; the hyperparameter sweep in Figure 3 shows $K \in \{4, 8, 16\}$ forms a shallow plateau with $K=8$ numerically strongest.

(2) The advantage persists out-of-domain: only CSM $K=8$ improves over FULL-ATTN on BEIR (+0.65; Appendix Table 17; trained on code, evaluated on natural-language BEIR with no fine-tuning), while all other compressed designs fall below it, confirming that the compression locus shapes retrieval quality beyond the training domain.

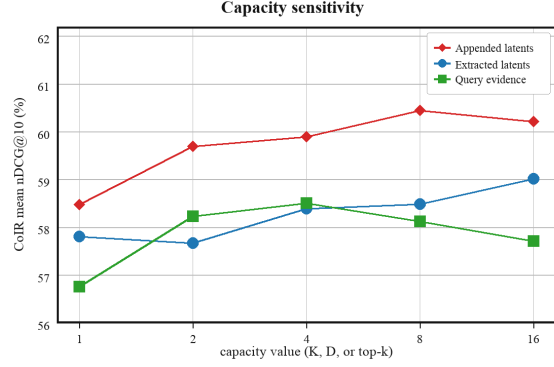


Figure 3: Hyperparameter sensitivity for source-exposure capacity on the completed no-vnorm seed-42 support sweeps. The in-source CSM curve is stable around $K = 4..16$ and peaks at $K = 8$; exact values are reported in Appendix Table 19.

CSM is therefore not an arbitrary compression trick: placing compression inside the source self-attention pass specifically improves the signal reaching the retriever’s score channel.

SGR correctness and scalability. We separate SGR evidence into completed gradient equivalence and the completed g256 endpoint; streaming-memory and intermediate pool-size sweeps remain attribution checks. (1) Full SGR passes the 10^{-6} acceptance threshold on all gradient paths (Appendix Table 4); no-replay and source-memory-only replay both fail on the routing-score or retriever path, confirming that both the source-memory and routing-score replay paths are necessary for gradient correctness.

(2) Table 3 and Figure 2 show that SGR supports the target 256-doc single-node pool with 52.95 GB peak memory and full 10-task CoIR evaluation. The remaining streaming-memory, intermediate-GPU, and intermediate-pool sweeps are system-attribution measurements rather than additional completed quality endpoints.

SGR is therefore a gradient-correct mechanism for data-parallel NTP-supervised training; the remaining measurements determine how much of the full-system gain should be attributed to streaming memory versus larger-pool supervision.

6 Discussion

Scope. The main claim is architectural and in-domain. Under matched 1B NTP training on the Revela code corpus, CSM at $K=8$ is the strongest source representation tested and lowers reader-

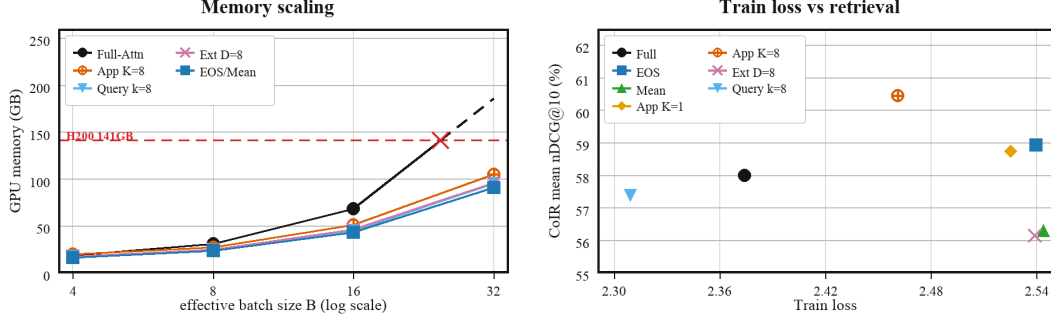


Figure 4: Memory and loss diagnostics. Left: 0.5B memory scaling with a $\log-B$ x-axis; the horizontal line marks single-H200 memory and the red cross marks where extrapolated FULL-ATTN exceeds it. Right: train loss versus auxiliary CoIR quality.

side memory. The full-system CSGR claim additionally requires the matched global-pool ablation: stop-gradient cache without replay, CSGR with CSM+SGR, and a full-token global feasibility control. SGR’s local gradient equivalence is verified in a deterministic fp32 test, but the larger system claims still require streaming-memory and global-pool scaling measurements. We therefore avoid multi-node claims and state the scaling target as single-node distributed global-pool training.

Broader impact and ethics. This method may reduce reliance on labeled retrieval data, synthetic query generation, and large teacher rankers, but it can also improve retrieval over unsafe, outdated, copyrighted, privacy-sensitive, or biased corpora. Because NTP-induced updates can reward superficial continuation cues, high retriever scores should not be treated as correctness or safety guarantees. We limit claims to offline public-benchmark experiments and document asset provenance and licenses in Appendix H; the work does not involve human-subject experiments, crowdsourcing, or collection of new personal data.

7 Conclusion

NTP-style retriever training compresses all reader-side utility into one document-level score gradient per candidate, so both the source interface and the gradient path shape what survives into the retriever update. CSGR makes this training regime scalable by pairing CSM’s compact source memories with SGR’s gradient-correct replay. The current completed experiments show that CSM $K=8$ preserves or improves retrieval quality while reducing memory: it improves mean CoIR nDCG@10 by +1.17 over full cross-document attention in

the 1B memory-backed setting (52.3 vs 72.3 GB), preserves the direction in the 3B scale-up check with a +1.93 gain, and passes a deterministic SGR gradient-equivalence test. The final experimental closure is now focused: the full system must beat the matched stop-gradient global-pool control, and the streaming-memory and global-pool scaling measurements must complete the system evidence.

Limitations

Our study is scoped to NTP-style self-supervised retriever training on the Revela code corpus, with CoIR as the primary in-domain evaluation and BEIR as a single-seed direction check. The reader is restricted to Llama-3.2 at 1B and 3B, and the retriever to a LoRA-tuned variant of the same backbone; whether the observed source-exposure ordering transfers to larger readers, encoder-only retrievers, or non-code corpora is left open. Compact in-source latent capacity is examined over $K \in \{4, 8, 16\}$ and may interact with sequence length, candidate-set size, or routing temperature outside the swept range. BEIR results are reported at one seed and should be read as a direction check rather than a matched headline. All claims are restricted to offline public-benchmark experiments; we make no operational claims about online retrieval quality, safety, or robustness on unsafe, copyrighted, or privacy-sensitive corpora.

References

- Luiz Bonifacio, Hugo Abonizio, Marzieh Fadaee, and Rodrigo Nogueira. 2022. [Inpars: Data augmentation for information retrieval using large language models](#). *Preprint*, arXiv:2202.05144.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann,

601	Trevor Cai, Eliza Rutherford, Katie Millican, George	Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick	655
602	van den Driessche, Jean-Baptiste Lespiau, Bogdan	Lewis, Ledell Wu, Sergey Edunov, Danqi Chen,	656
603	Damoc, Aidan Clark, Diego de Las Casas, Aure-	and Wen-tau Yih. 2020. Dense passage retrieval	657
604	lia Guy, Jacob Menick, Roman Ring, Tom Hennig-	for open-domain question answering . <i>Preprint</i> ,	658
605	gan, Saffron Huang, Loren Maggiore, Chris Jones,	arXiv:2004.04906.	659
606	Albin Cassirer, and 9 others. 2022. Improving lan-		
607	guage models by retrieving from trillions of tokens .	Juho Lee, Yoonho Lee, Jungtaek Kim, Adam R. Ko-	660
608	<i>Preprint</i> , arXiv:2112.04426.	sioerek, Seungjin Choi, and Yee Whye Teh. 2019.	661
609	Mikhail S. Burtsev, Yuri Kuratov, Anton Peganov, and	Set transformer: A framework for attention-based	662
610	Grigory V. Sapunov. 2020. Memory transformer .	permutation-invariant neural networks . <i>Preprint</i> ,	663
611	<i>Preprint</i> , arXiv:2006.11527.	arXiv:1810.00825.	664
612	Fengyu Cai, Tong Chen, Xinran Zhao, Sihao Chen,	Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio	665
613	Hongming Zhang, Sherry Tongshuang Wu, Iryna	Petroni, Vladimir Karpukhin, Naman Goyal, Hein-	666
614	Gurevych, and Heinz Koepl. 2026. Revela: Dense	rich Küttler, Mike Lewis, Wen-tau Yih, Tim Rock-	667
615	retriever learning via language modeling . <i>Preprint</i> ,	täschel, Sebastian Riedel, and Douwe Kiela. 2021.	668
616	arXiv:2506.16552.	Retrieval-augmented generation for knowledge-	669
617		intensive nlp tasks . <i>Preprint</i> , arXiv:2005.11401.	670
618	Zhuyun Dai, Vincent Y. Zhao, Ji Ma, Yi Luan, Jianmo	Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning:	671
619	Ni, Jing Lu, Anton Bakalov, Kelvin Guu, Keith B.	Optimizing continuous prompts for generation . In	672
620	Hall, and Ming-Wei Chang. 2023. Promptagator:	<i>Proceedings of the 59th Annual Meeting of the Asso-</i>	673
621	Few-shot dense retrieval from 8 examples . In <i>The</i>	<i>ciation for Computational Linguistics</i> , pages 4582–	674
622	<i>Eleventh International Conference on Learning Rep-</i>	4597.	675
623	<i>resentations</i> . OpenReview.net.	Xiangyang Li, Kuicai Dong, Yi Quan Lee, Wei Xia,	676
624	Luyu Gao, Yunyi Zhang, Jiawei Han, and Jamie Callan.	Hao Zhang, Xinyi Dai, Yasheng Wang, and Ruim-	677
625	2021. Scaling deep contrastive learning batch size	ing Tang. 2025. Coir: A comprehensive bench-	678
626	under memory limited setup . In <i>Proceedings of the</i>	mark for code information retrieval models . <i>Preprint</i> ,	679
627	<i>6th Workshop on Representation Learning for NLP</i> .	arXiv:2407.02883.	680
628	Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasu-	Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar,	681
629	pat, and Ming-Wei Chang. 2020. Realm: Retrieval-	and Timothy P. Lillicrap. 2019. Compressive trans-	682
630	augmented language model pre-training . <i>Preprint</i> ,	formers for long-range sequence modelling . <i>Preprint</i> ,	683
631	arXiv:2002.08909.	arXiv:1911.05507.	684
632	Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and	Weijia Shi, Sewon Min, Michihiro Yasunaga, Min-	685
633	Ross Girshick. 2020. Momentum contrast for un-	joon Seo, Rich James, Mike Lewis, Luke Zettle-	686
634	supervised visual representation learning . In <i>Pro-</i>	moyer, and Wen-tau Yih. 2023. Replug: Retrieval-	687
635	<i>ceedings of the IEEE/CVF Conference on Computer</i>	augmented black-box language models . <i>Preprint</i> ,	688
636	<i>Vision and Pattern Recognition</i> .	arXiv:2301.12652.	689
637	Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebas-	Nandan Thakur, Nils Reimers, Andreas Rücklé, Ab-	690
638	tian Riedel, Piotr Bojanowski, Armand Joulin, and	hishek Srivastava, and Iryna Gurevych. 2021. Beir:	691
639	Edouard Grave. 2022a. Unsupervised dense infor-	A heterogenous benchmark for zero-shot evalua-	692
640	mation retrieval with contrastive learning . <i>Preprint</i> ,	tion of information retrieval models . <i>Preprint</i> ,	693
641	arXiv:2112.09118.	arXiv:2104.08663.	694
642	Gautier Izacard and Edouard Grave. 2021. Distilling	Kexin Wang, Nandan Thakur, Nils Reimers, and Iryna	695
643	knowledge from reader to retriever for question	Gurevych. 2022a. GPL: Generative pseudo labeling	696
644	answering . In <i>9th International Conference on Learn-</i>	for unsupervised domain adaptation of dense retrieval .	697
645	<i>ing Representations</i> . OpenReview.net.	In <i>Proceedings of the 2022 Conference of the North</i>	698
646	Gautier Izacard, Patrick Lewis, Maria Lomeli, Lu-	<i>American Chapter of the Association for Computa-</i>	699
647	cas Hosseini, Fabio Petroni, Timo Schick, Jane	<i>tional Linguistics: Human Language Technologies</i> ,	700
648	Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and	pages 2345–2360. Association for Computational	701
649	Edouard Grave. 2022b. Atlas: Few-shot learning	Linguistics.	702
650	with retrieval augmented language models . <i>Preprint</i> ,	Liang Wang, Nan Yang, Xiaolong Huang, Binx-	703
651	arXiv:2208.03299.	ing Jiao, Linjun Yang, Daxin Jiang, Rangan Ma-	704
652	Andrew Jaegle, Felix Gimeno, Andrew Brock, Andrew	jumder, and Furu Wei. 2022b. Text embeddings by	705
653	Zisserman, Oriol Vinyals, and Joao Carreira. 2021.	weakly-supervised contrastive pre-training . <i>Preprint</i> ,	706
654	Perceiver: General perception with iterative attention .	arXiv:2212.03533.	707
	<i>Preprint</i> , arXiv:2103.03206.	Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas	708
		Muennighoff. 2023. C-pack: Packaged resources	709
		to advance general chinese embedding . <i>Preprint</i> ,	710
		arXiv:2309.07597.	711

A Gradient Equivalence Verification

Proof of Proposition 1. *Source-memory path.* By the chain rule, $\nabla_{\theta_s} \mathcal{L} = \sum_j (\sum_{i: j \in \mathcal{S}_i} \partial \mathcal{L}_i / \partial M_j) \cdot \partial M_j / \partial \theta_s$. SGR’s accumulation step computes exactly $G_j = \sum_{i: j \in \mathcal{S}_i} \partial \mathcal{L}_i / \partial \hat{M}_j$; since $\hat{M}_j$ and M_j agree in forward value, the replay step applies $G_j \cdot \partial M_j / \partial \theta_s$, recovering the full-batch chain-rule expression. *Routing-score path.* $\partial \mathcal{L} / \partial s_{ij} = \sum_i \partial \mathcal{L}_i / \partial s_{ij}$. In SGR the routing weights α_{ij} are live during the stream step; accumulating per-target score gradients and summing equals the full-batch derivative by linearity of differentiation. $\square$

Numerical verification. We verify Proposition 1 numerically with a deterministic fp32 sanity test generated by `scripts/smoke_scgr_gradient_equivalence.py`; the raw JSON artifact is `docs/generated/scgr_gradient_equivalence_20260516_ge26_29.json`. The module uses four batch documents, input dimension 16, hidden size 32, memory dimension 8, and one compression-token equivalent. The query and passage encoders are linear maps followed by normalization; the source compressor is a linear map followed by `tanh`; routing scores are query–passage dot products; routing weights are softmax-normalized; the reader maps the weighted source memory to a scalar MSE loss.

The reference row keeps the full autograd graph. The no-replay row detaches the cached source memory and routing path. The source-memory-only row replays the compressor path but not the retriever/routing path. The full-SGR row replays both paths. The pass/fail thresholds for the full-SGR row are 10^{-6} max absolute error for loss, routing-score gradients, retriever gradients, and source-compressor gradients, plus the requirement that source-compressor gradients are present.

This test verifies the local cache-boundary gradient math, not end-to-end mixed-precision distributed equality. It should therefore be read as correctness evidence for SGR’s replay identity; quality, throughput, and distributed numerical tolerance are measured separately.

B Experiment Map

The appendix keeps the detailed evidence behind the main-text experiments. It follows one

chain rather than a flat experiment dump. First, Appendix E and Appendix G define the exposure operators, reader-side cost accounting, and matched training/evaluation protocol. Second, Appendix J.1 reports the completed CSM memory, scale, and reader-loss diagnostics used in the source-representation analysis. Third, Appendix L and Appendix M give support sweeps and mined case studies that explain where compact exposure helps and where FULL-ATTN still helps. The full-system and SGR tables remain in the main text so the remaining experimental cells are explicit.

Table 4: Detailed gradient-equivalence smoke test. Each row is compared against the full-graph reference; “missing” means the detached-cache variant produces no routing-score gradient at the measured boundary.

Metric	FULLGRAPH	No replay	Source-memory only	Full SGR
Summary	reference	routing missing; retriever err 1.49×10^{-2} ; CSM grad missing/wrong	CSM err 0.0; routing missing; retriever err 1.49×10^{-2}	all measured errors 0.0
Loss abs diff	0.0	0.0	0.0	0.0
Routing-score grad err	0.0	missing	missing	0.0
Retriever grad err	0.0	0.0149406325	0.0149406325	0.0
CSM/source-compressor grad err	0.0	0.3857972026	0.0	0.0
Compressor grad present	true	false	true	true
Status	reference	paper-ready ablation	paper-ready ablation	paper-ready

Table 5: Appendix result inventory.

Block	Claim support
Protocol/operators	Matched source-exposure definitions and reproducibility protocol (Appendix C; Tables 6–7; Appendix G).
Representation	CSM $K=8$ preserves or improves quality over full-token exposure and compact controls (Table 2).
Cost/scale/loss	Memory, loss, and runtime diagnostics support the quality–memory trade-off claim without replacing endpoint estimates (Figures 7–8; Tables 14, 15, 16).
Capacity sweeps	Capacity and routed-layer sweeps show moderate CSM capacity is sufficient and timing-sensitive (Figure 9; Tables 19–22).
Full system / SGR	Joint ablation plan, gradient equivalence, and remaining system-attribution measurements for full-system and SGR claims (Table 3; Table 4).
Case studies	Mined disagreements show where exact local evidence helps FULL-ATTN and where surface-overlap cues are spurious (Tables 24–26; Appendix M.2).

C Expanded Source-Exposure Bookkeeping

This section gives the longer bookkeeping that was compressed out of the main text. Let $\mathcal{A}$ denote one source-exposure pattern. It specifies the readable memory exposed by each source document and the target-side computation that reads that memory. We write the exposed memory as

$$M_j^{\ell, \mathcal{A}} = \mathcal{M}_{\mathcal{A}}^{\ell}(D_j)$$

and the reader contribution as

$$\Delta H_{ij}^{\ell, \mathcal{A}} = \mathcal{C}_{\mathcal{A}}^{\ell}(H_i^{\ell}, M_j^{\ell, \mathcal{A}}).$$

At routed layers, the target state is updated as

$$\tilde{H}_i^{\ell} = H_i^{\ell} + \gamma_{\ell} \sum_{j \in \mathcal{S}_i} \alpha_{ij} \Delta H_{ij}^{\ell, \mathcal{A}}, \quad \ell \in \mathcal{R}, \quad (6)$$

where $\mathcal{R}$ is the routed-layer set and γ_{ℓ} is the injection strength at layer ℓ . Changing $\mathcal{A}$ leaves the nominal NTP objective and routing rule intact, but changes the exposed memories $M_j^{\ell, \mathcal{A}}$ and the reader contribution $\Delta H_{ij}^{\ell, \mathcal{A}}$ that can produce NTP utility.

Different source-exposure patterns instantiate $M_j^{\ell, \mathcal{A}}$ differently. FULL-ATTN exposes full source-token key-value states (K_j^{ℓ}, V_j^{ℓ}) . EOS STATE exposes $(K_{j, \text{eos}}^{\ell}, V_{j, \text{eos}}^{\ell})$. MEAN STATE exposes averaged key-value states. APPENDED LATENTS exposes the appended latent slice

$$\text{slice}_{L+1:L+K}(\text{KV}^{\ell}(D_j, c_1, \dots, c_K)),$$

where $c_1, \dots, c_K$ are learned latent positions appended to the source pass. EXTRACTED LATENTS exposes

$$\text{KV}^{\ell}(\text{Attn}(Q_{\text{lat}}^{\ell}, H_j^{\ell}, H_j^{\ell})),$$

where learned latent queries extract D source-conditioned memories. QUERY EVIDENCE exposes selected source-token key-value states. These cases differ in what the language model can read from each candidate before token-level utility is collapsed into document-level feedback.

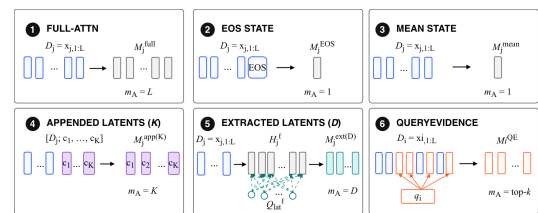


Figure 5: Source-exposure patterns organized by compression locus and learnability. FULL-ATTN exposes raw source-token states; APPENDED LATENTS forms compact states inside the source self-attention pass; EXTRACTED LATENTS forms compact states outside it; EOS STATE and MEAN STATE use non-learned summaries, and QUERY EVIDENCE uses non-parametric token selection. All six pass through the score channel of Eq. 3.

D Score-Channel Derivation

For a fixed target, checkpoint, candidate set, and source-exposure pattern, write the routed language-

modeling loss as

$$\mathcal{L}_{i,\mathcal{A}} = - \sum_t \log p_\phi(x_{i,t} \mid x_{i,<t}, \{\tilde{H}_i^\ell\}_{\ell \in \mathcal{R}}).$$

Define the uncentered NTP utility of source D_j as

$$U_{ij}^{\mathcal{A}} = -\partial \mathcal{L}_{i,\mathcal{A}} / \partial \alpha_{ij}.$$

If $U_{ij}^{\mathcal{A}}$ is large, locally allocating more routing mass to D_j would lower the NTP loss more. Let $G_i^\ell = \partial \mathcal{L}_{i,\mathcal{A}} / \partial \tilde{H}_i^\ell$ be the loss gradient at routed layer ℓ . From Eq. 6, the utility can be expanded as

$$\begin{aligned} U_{ij}^{\mathcal{A}} &= - \sum_{\ell \in \mathcal{R}} \gamma_\ell \langle G_i^\ell, \Delta H_{ij}^{\ell,\mathcal{A}} \rangle \\ &= - \sum_{\ell \in \mathcal{R}} \sum_t \gamma_\ell \langle G_{i,t}^\ell, \Delta H_{ij,t}^{\ell,\mathcal{A}} \rangle. \end{aligned} \quad (7)$$

Thus token-level and layer-level reader contributions from a candidate document are compressed into one document-level scalar $U_{ij}^{\mathcal{A}}$.

For fixed i and $\mathcal{A}$, write $\alpha_j = \alpha_{ij}$, $s_j = s_{ij}$, and $U_j = U_{ij}^{\mathcal{A}}$. The routing weights are

$$\alpha_k = \frac{\exp(s_k/\tau)}{\sum_{m \in \mathcal{S}_i} \exp(s_m/\tau)}. \quad (8)$$

For $j, k \in \mathcal{S}_i$,

$$\frac{\partial \alpha_k}{\partial s_j} = \frac{1}{\tau} \alpha_k (\mathbf{1}[j = k] - \alpha_j). \quad (9)$$

Using $U_k = -\partial \mathcal{L}_{i,\mathcal{A}} / \partial \alpha_k$ and the chain rule,

$$\frac{\partial \mathcal{L}_{i,\mathcal{A}}}{\partial s_j} = \sum_{k \in \mathcal{S}_i} \frac{\partial \mathcal{L}_{i,\mathcal{A}}}{\partial \alpha_k} \frac{\partial \alpha_k}{\partial s_j} \quad (10)$$

$$= -\frac{1}{\tau} \sum_{k \in \mathcal{S}_i} U_k \alpha_k (\mathbf{1}[j = k] - \alpha_j) \quad (11)$$

$$= -\frac{1}{\tau} \alpha_j \left(U_j - \sum_{k \in \mathcal{S}_i} \alpha_k U_k \right). \quad (12)$$

Defining the centered document credit $C_{ij}^{\mathcal{A}} = U_{ij}^{\mathcal{A}} - \mathbb{E}_{k \sim \alpha_i}[U_{ik}^{\mathcal{A}}]$, we obtain

$$\begin{aligned} g_{ij}^{\mathcal{A}} &\triangleq -\frac{\partial \mathcal{L}_{i,\mathcal{A}}}{\partial s_{ij}} \\ &= \frac{1}{\tau} \alpha_{ij} (U_{ij}^{\mathcal{A}} - \mathbb{E}_{k \sim \alpha_i}[U_{ik}^{\mathcal{A}}]) \\ &= \frac{1}{\tau} \alpha_{ij} C_{ij}^{\mathcal{A}}. \end{aligned} \quad (13)$$

This is a local accounting identity for any softmax-routed differentiable loss; it does not by itself

explain why one exposure pattern outperforms another. Applying the chain rule once more through $s_{ij} = s_{\theta_r}(D_i, D_j)$ gives the corresponding retriever-parameter gradient while holding the selected candidate set fixed:

$$\begin{aligned} \nabla_{\theta_r} \mathcal{L}_{i,\mathcal{A}} &= \sum_{j \in \mathcal{S}_i} \frac{\partial \mathcal{L}_{i,\mathcal{A}}}{\partial s_{ij}} \nabla_{\theta_r} s_{ij} \\ &= - \sum_{j \in \mathcal{S}_i} g_{ij}^{\mathcal{A}} \nabla_{\theta_r} s_{ij}. \end{aligned} \quad (14)$$

No matter how many source tokens, attention heads, or reader interactions contribute to token prediction, the retriever receives supervision through one scalar score channel per candidate document.

The scope is deliberately local. We fix the selected candidates, detach top- K_s retrieval and corpus construction, and assume finite temperature $\tau > 0$ and differentiability of the reader with respect to routing weights. The derivation does not prove convergence, does not claim differentiability through retrieval, and does not explain why a bottleneck size such as $K=8$ should be structurally optimal. Its role is to identify the score channel that constrains the retriever update.

Exposure as a controlled intervention. FULL-ATTN lets the reader access all source-token states. This can improve reader-side evidence capacity, but it does not increase the dimensionality of the retriever update. The retriever is not told that one token was useful, another was incidental, and another carried the real evidence; it is told only whether the document score should increase or decrease relative to other candidates. Thus full token access and fine-grained retriever supervision are different things.

Changing $\mathcal{A}$ changes $U_{ij}^{\mathcal{A}}$, and therefore changes the centered document credit received by the retriever under the same candidates and routing rule. A simple construction illustrates why this intervention is measurable. Consider two exposure patterns $\mathcal{A}$ and $\mathcal{B}$, a target D_i , and two candidate documents D_j, D_k with the same routing scores and NTP objective, but with $U_{ij}^{\mathcal{A}} > U_{ik}^{\mathcal{A}}$ and $U_{ij}^{\mathcal{B}} < U_{ik}^{\mathcal{B}}$. One candidate may contain local continuation cues that help NTP under one exposure pattern; another may contain evidence that better supports the retrieval relation but is only useful when preserved by a different pattern. This is not a predictive theorem; it is the measurement motivation for comparing source-exposure operators.

E Reader-Side Exposure Cost Details

Source exposure also controls the routed reader’s variable memory footprint. Let B be target sequences per device, K_s candidate sources per target, $|\mathcal{R}|$ routed layers, T target positions participating in routed reading, d_{model} hidden width, and $m_{\mathcal{A}}$ the number of readable states exposed per source under $\mathcal{A}$. Absorbing constants for precision, heads, and forward/backward storage,

$$M_{\mathcal{A}} \approx M_{\text{base}} + c_{\text{kv}} B K_s |\mathcal{R}| m_{\mathcal{A}} d_{\text{model}} + c_{\text{attn}} B K_s |\mathcal{R}| T m_{\mathcal{A}}, \quad (15)$$

with $m_{\mathcal{A}} = L$ for FULL-ATTN, K for APPENDED LATENTS, D for EXTRACTED LATENTS, k_{tok} for QUERY EVIDENCE, and 1 for EOS STATE/MEAN STATE. Eq. 15 describes only the exposure-dependent term; measured peak memory in Section 5 also reflects model, optimiser, ZeRO partitioning, non-routed activations, source-side extraction overhead, and allocator reserve, which are why the empirical $\sim 28\%$ memory gap is much smaller than the nominal L/K ratio.

Table 7 expands Eq. 15 by exposure operator. The table tracks the exposure-dependent routed-reader terms, not the full training memory model. In measured GPU runs, the observed peak also includes model parameters, optimizer state, ZeRO partitioning, non-routed activations, source-encoder computation, framework workspaces, and CUDA allocator reserve.

F External CoIR Baseline Details

Table 8 reports the external task-level CoIR baselines used only for context; they are not re-run or mixed into the matched internal ablations.

G Implementation details

For reproducibility, the implementation record includes tokenizer, source length L , backbone LM, retriever encoder, optimizer, learning rate schedule, temperature τ , training steps, hardware, checkpoint selection, random seeds, and evaluation commands. Run-name conventions and matched-run provenance are summarized below.

Environment. The main local environment uses Python 3.8.20, PyTorch 2.4.1+cu121, Transformers 4.45.2, DeepSpeed 0.15.2, PEFT 0.13.2, Datasets 3.1.0, MTEB 1.14.22, coir-eval 0.7.0, and pytrac-eval-terrier 0.5.6. Training uses DeepSpeed ZeRO-3, bf16, and $8 \times 80\text{GB}$ GPUs.

The anonymous artifact includes the Conda environment file, DeepSpeed config, source patch, and sanitized launch scripts.

Data and splits. Training uses the Revela code training corpus, a JSONL file of 358,763 batches built from CodeRAG-Bench StackOverflow posts, online tutorials, and library documentation. The local checksum for the training JSONL used by the reproduction package is:

```
3ca23d7d70e75fc674851e1b259a5d4d
83bb4aac48130f42af513b205aa9acb7
```

CoIR labels are not used during training. Evaluation uses the official CoIR query, corpus, and qrel files for the ten tasks in Table 2; qrels are used only for nDCG@10 evaluation and post-hoc case-study grouping.

Training command template. All source-exposure rows share the command template below; the exposure-specific variables are listed after the block. Absolute paths are anonymized here, and no API keys or private logging credentials are required for reproduction.

Matched endpoint comparisons repeat training with $-\text{seed} \in \{42, 43, 44\}$; the block above shows $-\text{seed} 42$ as an example.

The main operator settings are: FULL-ATTN uses SOURCE_MEMORY_MODE=full, COMPRESSION_TOKENS=0, and DETAIL_SLOT_COUNT=0, and QUERY_EVIDENCE_MODE=off; EOS STATE uses SOURCE_MEMORY_MODE=eos; MEAN STATE uses SOURCE_MEMORY_MODE=mean; APPENDED LATENTS uses SOURCE_MEMORY_MODE=compression with COMPRESSION_TOKENS=K; EXTRACTED LATENTS uses SOURCE_MEMORY_MODE=detail with DETAIL_SLOT_COUNT=D; and QUERY EVIDENCE top- k uses SOURCE_MEMORY_MODE=full, QUERY_EVIDENCE_MODE=topk_full_kv, and QUERY_TOKEN_TOP_K=k.

Checkpointing and evaluation. Runs with $b=4, d=4, \text{ga}=4$ train for one epoch, 2803 optimizer steps in the current code-corpus setup, saving every 500 steps and at the final checkpoint. CoIR evaluation encodes queries and corpus with batch size 64, evaluates the ten CoIR tasks independently, and reports the unmerged clean nDCG@10 mean only when all ten task summaries are present. The qualitative case study uses completed CoIR rank-

Table 6: Source-exposure patterns compared in the matched CoIR line.

Operator	Source memory	Size	Role
FULL-ATTN	full token KV	L	maximum-access baseline
APPENDED LATENTS	appended latent KV	default $K = 8$	appended-memory exposure probe
EXTRACTED LATENTS	extracted latent memories	$D = 8$	learned-extraction exposure probe
EOS STATE	EOS-position KV	1	single-state compression control
MEAN STATE	mean-pooled state	1	pooling control
QUERY EVIDENCE	selected tokens	$k_{\text{tok}} = 8$	evidence-token control

Table 7: Reader-side exposure complexity. Constants for precision, heads, and forward/backward storage are omitted.

Operator	m_A	Routed target-source term	Extra source-side overhead
FULL-ATTN	L	$O(BK_s \mathcal{R} TL)$	Stores full source-token KV states for routed reading.
EOS STATE	1	$O(BK_s \mathcal{R} T)$	Selects the EOS-position state; no learned extraction step.
MEAN STATE	1	$O(BK_s \mathcal{R} T)$	Pools source states before routing; reduction over L source positions is still computed.
APPENDED LATENTS (K)	K	$O(BK_s \mathcal{R} TK)$	Runs the source pass with K appended learned latent positions and stores their KV states.
EXTRACTED LATENTS (D)	D	$O(BK_s \mathcal{R} TD)$	Learned latent queries extract D memories from source states, adding an extraction attention term such as $O(BK_s \mathcal{R} DL)$.
QUERY EVIDENCE token selection	k_{tok}	$O(BK_s \mathcal{R} T k_{\text{tok}})$	Scores or selects evidence tokens over the source and gathers selected KV states; selection temporaries are not free.

ings and qrel-positive ranks post hoc; the scripts that mine disagreement buckets are included in the artifact.

Open release plan. The anonymous supplement includes code modifications, training and evaluation scripts, environment files, DeepSpeed config, run metadata schema, sanitized command templates, and generated case-study metadata. It does not redistribute Llama weights, raw third-party training data, private checkpoints, W&B keys, or machine-local paths. Users must obtain gated base models and public datasets under their original terms.

H Assets, Licenses, and Terms

Table 10 summarizes the external assets used in the paper. The anonymous supplement releases only our code modifications, scripts, metadata schemas, and derived evaluation or case-study metadata needed to reproduce the analyses. It does not re-host gated model weights, raw benchmark corpora, raw training corpora, or proprietary API outputs. When an asset aggregates multiple upstream sources, the table records the aggregate package when available and notes that the original source terms still apply.

I Run provenance

Old experiment directory names may contain `chorus` or `qchorus`; those names are retained only for reproducibility and do not define the paper terminology. In method suffixes, `d0` means zero

extracted-latent slots, `k0` means zero appended-latent slots, and `r0` usually denotes all interaction layers rather than zero interaction layers, unless the table provenance gives an explicit routed-layer count. When `latehalf` is present, the `r8` suffix denotes eight late-half interaction layers. In training suffixes, `b` is the number of target samples per GPU, `d` is the number of documents carried with each target sample, and `ga` is the number of gradient-accumulation steps; thus a suffix such as `b4-ga2-d4` corresponds to $b=4, d=4, ga=2$, with per-GPU target-source batch size $b \times d = 16$ and two accumulation steps. All internal columns in Appendix J should be read as clean evaluation rows for an exposure operator; run ids, checkpoint numbers, and implementation flags identify provenance rather than additional method variants. The three endpoint columns in Table 2 are the complete matched $b=4, d=4, ga=4$ main line with clean CoIR evaluations; auxiliary appendix rows are not used to estimate the main +1.17 CoIR claim. In the no-vnorm support rows, `no-vnorm` means the implementation’s `vnorm` flag is disabled, `444` denotes the $b=4, d=4, ga=4$ profile, `checkpoint 2803` is the evaluated checkpoint, and `latest_eval_bucket` is the clean no-merge CoIR evaluation artifact. One concrete run id from the matched main line, the APPENDED LATENTS default $K = 8$ row, is:

```
revela-overnight-novnorm-1b-
summaryslots-k8-distinct-bidir-
alllayers-ga4-b4-d4-
0501-overnight-444-novnorm-safe3
```

1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037

1038

Table 8: Task-level external CoIR baselines (nDCG@10, %). Published baseline scores are reproduced from (Li et al., 2025; Cai et al., 2026); we did not re-run external baselines. † marks models using code-related query-document pairs in supervised pre-training; APIs are evaluated zero-shot. The last column repeats our matched 1B APPENDED LATENTS ($K=8$) three-seed mean from Table 2. Bold marks the best score in each row.

Task	BM25	UniX [†] 0.1B	E5-PT [†] 0.3B	BGE [†] 0.6B	E5-Mistral [†] 7B	Ada-2	Voyage-2	Ours App. Latents $K=8$ (1B)
Apps Retrieval	4.8	1.4	10.6	14.7	23.5	8.7	26.5	19.55
CosQA	15.6	25.1	27.1	26.4	33.2	28.9	29.8	31.49
Synthetic Text2SQL	25.0	50.5	48.9	46.9	68.0	58.3	69.3	58.87
COIR CodeSearchNet	40.9	60.2	35.2	58.3	67.4	74.2	81.8	82.13
CodeSearchNet CC	54.0	58.4	50.5	53.7	64.8	53.3	72.8	75.09
CodeTransOcean Contest	47.8	41.8	56.3	62.6	80.6	26.0	27.3	75.97
CodeTransOcean DL	34.4	31.0	32.2	30.2	31.7	17.7	28.7	34.57
StackOverflow QA	70.2	44.7	86.9	80.6	91.0	74.2	81.8	88.37
CodeFeedback ST	68.1	36.0	70.4	69.3	76.4	47.1	65.4	76.52
CodeFeedback MT	59.2	24.2	46.2	47.9	36.4	17.7	28.7	57.08
Mean	42.0	37.3	46.4	49.1	57.3	45.6	56.3	59.96

Table 9: Training command template shared by source-exposure rows. Absolute paths are anonymized.

```

export REPO_ROOT=/path/to/anonymous/revela-source-exposure
export DATA_PATH=/path/to/revela_code_training_corpus/
training.jsonl
export MODEL_PATH=/path/to/meta-llama/Llama-3.2-1B
export REFERENCE_MODEL_PATH=/path/to/meta-llama/
Llama-3.2-1B
export OUTPUT_DIR=/path/to/output/${RUN_NAME}
export GPU_LIST=0,1,2,3,4,5,6,7
export WANDB_DISABLED=true

cd ${REPO_ROOT}
PYTHONPATH=${REPO_ROOT}/src \
deepspeed --include localhost:${GPU_LIST} \
--module tevatron.llm_retriever.driver.train \
--deepspeed deepspeed/ds_zero3_config.json \
--output_dir ${OUTPUT_DIR} \
--model_name_or_path ${MODEL_PATH} \
--reference_model_name_or_path ${REFERENCE_MODEL_PATH} \
--lora --lora_r 256 --lora_alpha 64 \
--lora_dropout 0.1 \
--lora_target_modules q_proj,k_proj,v_proj,o_proj \
--lora_target_modules down_proj,up_proj,gate_proj \
--bm25_retrieval_file ${DATA_PATH} \
--add_passage_prefix True --add_query_prefix True \
--passage_prefix "passage: " \
--query_prefix "query: " \
--first_half False --bf16 True --pooling eos \
--append_eos_token True \
--normalize True --temperature 0.01 \
--attn_temperature 0.0001 \
--exclude_diagonal True --disable_v_norm True \
--inbatch_attn_layer_strategy all \
--cross_attn_gate 1.0 \
--trainable_cross_attn_gate False \
--source_memory_mode ${SOURCE_MEMORY_MODE} \
--compression_tokens ${COMPRESSION_TOKENS} \
--detail_slot_count ${DETAIL_SLOT_COUNT} \
--legacy_query_evidence_mode ${QUERY_EVIDENCE_MODE} \
--query_token_top_k ${QUERY_TOKEN_TOP_K} \
--query_router_loss_beta 0.0 \
--query_router_temperature 1.0 \
--query_token_selection_mode heuristic_only \
--query_evidence_lambda 1.0 \
--per_device_train_batch_size 4 \
--train_group_size 4 \
--learning_rate 1e-4 --passage_max_len 157 \
--num_train_epochs 1 \
--gradient_accumulation_steps 4 \
--logging_steps 1 \
--warmup_steps 100 \
--data_loader_num_workers 4 --top_k 4 --seed 42 \
--save_steps 500 --run_name ${RUN_NAME} \
--report_to none \
--log_retriever_metrics True \
--log_system_metrics True \
--log_ntp_losses True

```

Table 10: External assets and release treatment.

Asset	Release and terms treatment
Code and local modifications	Release code, patches, scripts, configs, and metadata; remove secrets and machine-local paths.
Llama-3.2 backbones	Do not redistribute model weights; users obtain gated Llama access under Meta’s terms.
Revela training corpus	Provide path/checksum and preparation assumptions; do not redistribute raw corpus text.
CodeRAG-Bench	Users obtain upstream StackOverflow, tutorial, and documentation data under the original terms.
CoIR and coir-eval	Use official benchmark/qrels for evaluation; do not repack raw CoIR text or qrels.
External baselines	Reproduce only cited aggregate scores; do not redistribute baseline weights, API outputs, or proprietary responses.
BEIR	Use only for zero-shot evaluation; do not redistribute raw BEIR data.
Case-study metadata	Release ids, scripts, summaries, and derived metrics; omit raw copyrighted text unless permitted.

Table 11: Matched 3B CoIR scale-up check (nDCG@10, %). Results are mean $\pm$ standard deviation over seeds 42–44 on the clean 10-task CoIR evaluation. All rows use the same matched 3B setting; abbreviations follow the main text.

Exposure	3B CoIR mean
FULL-ATTN	59.95 $\pm$ 0.25
EOS STATE	59.35 $\pm$ 0.32
MEAN STATE	54.10 $\pm$ 0.10
App. Latents $K=8$	61.88$\pm$0.51
Ext. Latents $D=8$	60.64 $\pm$ 0.62
QueryEvidence $k=8$	59.33 $\pm$ 0.85

Table 12: Run provenance for the matched 3B scale-up check in Table 11. These rows support the endpoint ordering but are not used for the 80GB memory headline because the 3B FULL-ATTN row reserves about 91 GB.

Method	Provenance and CoIR mean
FULL-ATTN	3B; $b=4$, $ga=4$, $d=4$, $b \times d=16$; seeds 42–44; 10/10 tasks; 59.95 ± 0.25 .
EOS STATE	3B; $b=4$, $ga=4$, $d=4$, $b \times d=16$; seeds 42–44; 10/10 tasks; 59.35 ± 0.32 .
MEAN STATE	3B; $b=4$, $ga=4$, $d=4$, $b \times d=16$; seeds 42–44; 10/10 tasks; 54.10 ± 0.10 .
APPENDED LATENTS	3B; $b=4$, $ga=4$, $d=4$, $b \times d=16$; seeds 42–44; 10/10 tasks; 61.88 ± 0.51 .
EXTRACTED LATENTS	3B; $b=4$, $ga=4$, $d=4$, $b \times d=16$; seeds 42–44; 10/10 tasks; 60.64 ± 0.62 .
QUERY EVIDENCE	3B; $b=4$, $ga=4$, $d=4$, $b \times d=16$; seeds 42–44; 10/10 tasks; 59.33 ± 0.85 .

J Detailed Experiment Figures and Tables

This section contains the detailed numerical results behind the compact main-text presentation. We keep system diagnostics, capacity sweeps, and completed provenance rows here so that the main text can state the main conclusions without mixing matched ablations with support experiments. Tables 2 and 11 are the main endpoint tables; rows below are used only for loss, system-cost, capacity, or provenance diagnostics, not as alternate endpoint estimates for the 1B memory-backed headline.

Table 13: Internal source-representation ablation summary. CoIR is nDCG@10 mean $\pm$ standard deviation over seeds 42–44. APPENDED LATENTS $K=8$ is the default CSM configuration. The main text reports the task-level baseline comparison; this table keeps the compact-control rows visible.

Exposure	CoIR mean
FULL-ATTN	58.79 ± 0.04
EOS STATE	58.19 ± 0.34
MEAN STATE	52.99 ± 0.20
App. Latents $K=1$	58.90 ± 0.32
App. Latents $K=8$	59.96 ± 0.39
Ext. Latents $D=8$	57.05 ± 0.59
QueryEvidence $k=8$	57.90 ± 0.82

J.1 Reader-Loss and System Diagnostics

This block separates three quantities that are easy to conflate: token-level reader loss, diagnostic retrieval rows, and training footprint. Table 14 and

Figure 6 record the loss/retrieval mismatch on auxiliary rows; Table 15 records the main three-seed 1B endpoint and memory aggregation used by the abstract headline; Figure 7 and Table 16 record auxiliary single-run system costs with additional runtime fields; Figure 8 records the 0.5B batch-size memory sweep with the FULL-ATTN OOM endpoint extrapolated. The matched 3B endpoint check in Table 11 is reported separately from the 80GB memory-backed claim because the 3B FULL-ATTN row reserves about 91 GB. We separate these quantities because they answer different questions. The two memory tables intentionally use different aggregation levels: matched three-seed means for the memory-backed claim, and instrumented auxiliary rows for runtime diagnostics.

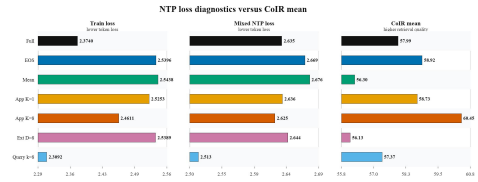


Figure 6: Log-extracted NTP/training losses versus CoIR retrieval. Lower token loss does not select the best exposure row.

Table 14: Key log-extracted NTP/training losses versus auxiliary CoIR retrieval rows. Lower token-level loss does not identify the best retrieval operator in these diagnostics.

Method	Train loss	Mixed NTP loss	CoIR mean
FULL-ATTN	2.3740	2.635	57.99
EOS STATE	2.5396	2.669	58.92
MEAN STATE	2.5438	2.676	56.30
APPENDED LATENTS ($K=1$)	2.5253	2.636	58.73
APPENDED LATENTS ($K=8$)	2.4611	2.625	60.45
EXTRACTED LATENTS ($D=8$)	2.5389	2.644	56.13
QUERY EVIDENCE top- $k=8$	2.3092	2.513	57.37

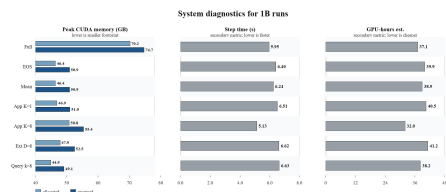


Figure 7: Cost-quality diagnostics for available auxiliary 1B runs on 8×80 GB GPUs.

Table 15: Matched 1B endpoint and memory summary used by the main-text contribution and Figure 4. CoIR is reported as mean $\pm$ standard deviation over seeds 42–44. Peak reserved memory is the mean rank-0 peak reserved CUDA memory over the same matched seed runs, so these values are not the auxiliary single-run diagnostics in Table 16.

Method	Endpoint and memory
FULL-ATTN	CoIR 58.79 ± 0.04 ; peak reserved 72.3GB; 90.4% of 80GB.
EOS STATE	CoIR 58.19 ± 0.34 ; peak reserved 48.2GB; 60.3% of 80GB.
APPENDED LATENTS ($K = 8$)	CoIR 59.96 ± 0.39 ; peak reserved 52.3GB; 65.4% of 80GB.

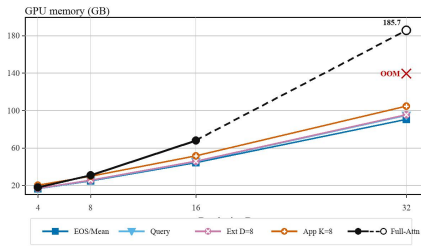


Figure 8: Batch-size memory scaling for the 0.5B seed-42 sweep. Completed points report rank-0 peak reserved CUDA memory; the red cross marks the FULL-ATTN $B = 32$ OOM allocation-plus-tried value for `gal-b4-d8`. The dashed endpoint extrapolates the completed FULL-ATTN–EOS STATE/MEAN STATE gap, approximately $0.093B^2$ GB.

Table 16: Cost-quality diagnostics for available auxiliary single-run 1B runs on 8×80 GB GPUs.

Method	CoIR and system diagnostics
FULL-ATTN	CoIR 57.99; 70.2GB alloc.; 74.7GB reserved; 93.4%; 5.95s/step; 37.1 GPU-h
EOS STATE	CoIR 58.92; 46.4GB alloc.; 50.9GB reserved; 63.6%; 6.40s/step; 39.9 GPU-h
MEAN STATE	CoIR 56.30; 46.4GB alloc.; 50.9GB reserved; 63.6%; 6.24s/step; 38.9 GPU-h
APPENDED LATENTS ($K = 1$)	CoIR 58.73; 46.9GB alloc.; 51.0GB reserved; 63.8%; 6.51s/step; 40.5 GPU-h
APPENDED LATENTS ($K = 8$)	CoIR 60.45; 50.8GB alloc.; 55.4GB reserved; 69.2%; 5.13s/step; 32.0 GPU-h
EXTRACTED LATENTS ($D = 8$)	CoIR 56.13; 47.9GB alloc.; 52.5GB reserved; 65.6%; 6.62s/step; 41.2 GPU-h
QUERY EVIDENCE top- $k = 8$	CoIR 57.37; 44.9GB alloc.; 49.1GB reserved; 61.3%; 6.63s/step; 38.2 GPU-h

K BEIR Generalization Provenance

The BEIR cross-domain summary in Table 17 is built from the same trained checkpoints used for the CoIR endpoint comparison: each source-exposure pattern is trained only on the Revela code training

Table 17: Cross-domain BEIR check (paper-13, nDCG@10 %). All rows are trained on the Revela code corpus and evaluated without BEIR-side fine-tuning.

Exposure	BEIR	Δ	CoIR
FULL-ATTN	42.93	–	58.79
EOS STATE	39.92	–3.01	58.19
MEAN STATE	39.29	–3.64	52.99
App. Latents $K=1$	37.28	–5.65	58.90
App. Latents $K=8$	43.58	+0.65	59.96
Ext. Latents $D=8$	39.82	–3.11	57.05
QueryEvidence $k=8$	42.85	–0.08	57.90

corpus and the resulting retriever is evaluated on the natural-language BEIR paper-13 suite without BEIR-side fine-tuning, following the cross-domain protocol used by prior NTP retriever work (Cai et al., 2026). Table 18 records per-row provenance (the seed-42 no-vnorm local union of completed BEIR evaluations) for reproducibility. For QUERY EVIDENCE top- $k = 8$, the previously missing entries on HotpotQA and FEVER are now filled (51.2020 and 63.4750 nDCG@10 respectively), giving the 13-task mean of 42.85 reported in the main text.

Table 18: BEIR paper-13 nDCG@10 snapshot. All rows report complete 13/13-task aggregates.

Method	Provenance and BEIR result
FULL-ATTN	current seed-42 no-vnorm local union; 13/13 tasks; BEIR mean 42.93; complete
EOS STATE	current seed-42 no-vnorm local union; 13/13 tasks; BEIR mean 39.92; complete
MEAN STATE	current seed-42 no-vnorm local union; 13/13 tasks; BEIR mean 39.29; complete
APPENDED LATENTS ($K = 1$)	current seed-42 no-vnorm local union; 13/13 tasks; BEIR mean 37.28; complete
APPENDED LATENTS ($K = 8$)	current seed-42 no-vnorm local union; 13/13 tasks; BEIR mean 43.58; complete
EXTRACTED LATENTS ($D = 8$)	current seed-42 no-vnorm local union; 13/13 tasks; BEIR mean 39.82; complete
QUERY EVIDENCE top- $k = 8$	current seed-42 no-vnorm local union; 13/13 tasks; BEIR mean 42.85; complete

L Support Sweeps and Additional Completed Runs

The following support sweeps answer narrower questions: whether compact-exposure capacity is monotonic, whether the current no-vnorm $b=4, d=4, ga=4$ profile has additional completed rows, and whether routed-layer timing affects results.

L.1 Completed no-vnorm support rows

Table 19 and Figure 9 report completed no-vnorm $b=8, d=4, ga=1$ support sweeps, all with seed

42. The table keeps both CoIR mean and Code-Feedback MT values for provenance, while the plot shows only CoIR mean so the capacity trend is not visually dominated by a single boundary task. These are single-seed support sweeps under a different batch profile. The corresponding $b=8, d=4, ga=1$ FULL-ATTN and EOS STATE baselines had run directories but no completed clean evaluation at the time of this update; no MEAN STATE baseline was found for this run stamp.

The sweep indicates a shallow plateau around $K = 4..16$ rather than a monotonic capacity trend. For APPENDED LATENTS, $K = 8$ is numerically highest, but its gain over $K = 4$ is only $+0.55$ CoIR and $K = 16$ is only 0.24 below $K = 8$, differences comparable to the 0.39 seed standard deviation of the main $K = 8$ endpoint.

Table 19: Completed no-vnorm $b=8, d=4, ga=1$, seed-42 support sweeps. Scores are nDCG@10 percentages from clean no-merge CoIR evaluations. CoIR mean is the quantity plotted in Figure 9; CodeFeedback MT is retained as a boundary-task diagnostic.

Family	Capacity	CoIR	CFB-MT
APPENDED LATENTS	$K = 1$	58.47	54.19
APPENDED LATENTS	$K = 2$	59.69	56.88
APPENDED LATENTS	$K = 4$	59.90	57.90
APPENDED LATENTS	$K = 8$	60.45	57.04
APPENDED LATENTS	$K = 16$	60.21	57.49
EXTRACTED LATENTS	$D = 1$	57.81	51.68
EXTRACTED LATENTS	$D = 2$	57.67	51.25
EXTRACTED LATENTS	$D = 4$	58.39	53.24
EXTRACTED LATENTS	$D = 8$	58.49	53.33
EXTRACTED LATENTS	$D = 16$	59.01	54.31
QUERY EVIDENCE	$top-k = 1$	56.76	54.95
QUERY EVIDENCE	$top-k = 2$	58.23	56.28
QUERY EVIDENCE	$top-k = 4$	58.51	55.82
QUERY EVIDENCE	$top-k = 8$	58.12	55.22
QUERY EVIDENCE	$top-k = 16$	57.71	52.20

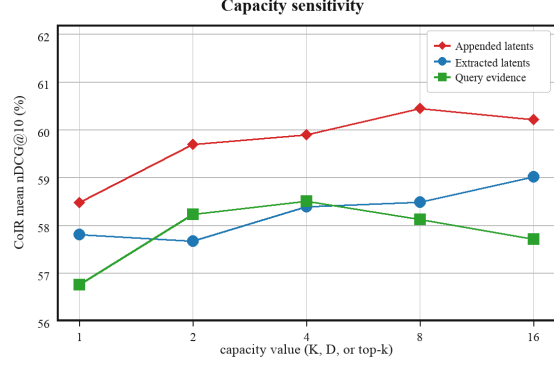


Figure 9: CoIR mean capacity sensitivity for the three completed no-vnorm seed-42 support sweeps. Code-Feedback MT is omitted from the plot to keep the figure focused on the aggregate endpoint trend; its values are reported in Table 19. Nearby APPENDED LATENTS values $K \in \{4, 8, 16\}$ are treated as an unresolved plateau rather than statistically distinguishable capacities.

Table 20 lists completed no-vnorm $b=4, d=4, ga=4$ support rows from the same current profile as the matched endpoint table. The rows here are reported only as capacity and nearby-control evidence, not as replacements for the main matched endpoint comparison.

Table 20: Completed $b=4, d=4, ga=4$ no-vnorm support rows. Scores are three-seed CoIR mean nDCG@10 percentages from clean latest_eval_bucket evaluations over seeds 42–44.

Family	Capacity	Seeds	CoIR mean
MEAN STATE	–	42–44	52.99 ± 0.20
APPENDED LATENTS	$K = 1$	42–44	58.90 ± 0.32
EXTRACTED LATENTS	$D = 8$	42–44	57.05 ± 0.59
QUERY EVIDENCE	$top-k = 8$	42–44	57.90 ± 0.82

The completed $b=4, d=4, ga=4$ no-vnorm rows are a current-profile support check. They show that the nearby support controls are stable across seeds, while remaining below the main APPENDED LATENTS default $K = 8$ endpoint mean. Because this table is an auxiliary capacity/control sweep, it is treated as support evidence rather than as a replacement for the matched endpoint table.

L.2 Legacy routed-layer sweeps

This appendix reports completed legacy CoIR ablations from the checked CSV. Scores are nDCG@10 percentages. These runs contextualize the source-exposure analysis, but they are not part of the matched endpoint table. In these sweeps, the run-id convention above applies unless the table gives explicit provenance: d0 disables extracted-latent

Table 21: Completed 1B APPENDED LATENTS CoIR sweeps. Training columns report per-GPU target batch size b , companion-document count d , gradient accumulation ga , and per-GPU target-source pairs $b \times d$. The R column reports the actual routed-layer count from provenance, not only the value implied by the run-name suffix.

Config	Metrics
$K = 4, D = 0, R = 4$, default	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 60.5166; CodeFeedbackST 76.8740; CodeFeedbackMT 56.7180
$K = 8, D = 0, R = 2$, default	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 58.9299; CodeFeedbackST 74.8910; CodeFeedbackMT 55.8350
$K = 8, D = 0, R = 0$, all layers	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 60.1098; CodeFeedbackST 77.5720; CodeFeedbackMT 58.1380
$K = 8, D = 0, R = 1$, default	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 58.4767; CodeFeedbackST 75.3300; CodeFeedbackMT 54.1410
$K = 8, D = 0, R = 4$, default	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 60.7768; CodeFeedbackST 77.3240; CodeFeedbackMT 55.9000
$K = 8, D = 0, R = 4$, contiguous early	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 51.9653; CodeFeedbackST 68.2270; CodeFeedbackMT 44.0680
$K = 8, D = 0, R = 4$, contiguous middle	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 57.2225; CodeFeedbackST 74.1470; CodeFeedbackMT 50.5920
$K = 8, D = 0, R = 4$, strided early	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 55.1866; CodeFeedbackST 71.6210; CodeFeedbackMT 44.5070
$K = 8, D = 0, R = 4$, strided middle	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 58.2628; CodeFeedbackST 74.4360; CodeFeedbackMT 53.2250
$K = 8, D = 0, R = 8$, default	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 60.6394; CodeFeedbackST 77.2330; CodeFeedbackMT 55.7060
$K = 16, D = 0, R = 4$, default	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 59.2845; CodeFeedbackST 76.3170; CodeFeedbackMT 55.5400

Table 22: Completed EXTRACTED LATENTS CoIR sweeps. All rows use the 1B backbone. Training columns follow Table 21.

Config	Metrics
1B, $D = 2, K = 0, R = 4$, default	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 58.8461; CodeFeedbackST 75.7850; CodeFeedbackMT 54.0140
1B, $D = 4, K = 0, R = 4$, default	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 57.8005; CodeFeedbackST 74.3940; CodeFeedbackMT 51.6060
1B, $D = 8, K = 0, R = 0$, all layers	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 58.1819; CodeFeedbackST 75.3810; CodeFeedbackMT 52.7180
1B, $D = 8, K = 0, R = 1$, default	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 54.9332; CodeFeedbackST 71.6870; CodeFeedbackMT 49.8760
1B, $D = 8, K = 0, R = 2$, default	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 55.7884; CodeFeedbackST 72.8260; CodeFeedbackMT 49.4000
1B, $D = 8, K = 0, R = 4$, default	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 58.7854; CodeFeedbackST 76.4750; CodeFeedbackMT 53.1750
1B, $D = 8, K = 0, R = 8$, default	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 57.0855; CodeFeedbackST 75.0080; CodeFeedbackMT 53.1620
1B, $D = 8, K = 0, R = 8$, late-half	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 58.6008; CodeFeedbackST 74.8240; CodeFeedbackMT 53.0070
1B, $D = 16, K = 0, R = 4$, default	$b = 4, ga = 2, d = 4, b \times d = 16$; CoIR 57.7358; CodeFeedbackST 75.2440; CodeFeedbackMT 52.3610

slots, `k0` disables appended-latent slots, and `r0` usually denotes all interaction layers.

The most useful pattern in these legacy sweeps is qualitative. For APPENDED LATENTS $K = 8$, early routed-layer interaction is weak (51.97 CoIR), while default or all-layer interactions are near 60–61 CoIR. This indicates sensitivity to routed-layer timing.

1144
1145
1146
1147
1148
1149
1150
1151

M Qualitative CoIR Case Study

This appendix gives the detailed case study behind the source-exposure interpretation. It is deliberately post-hoc: qrels are used only after the clean retrieval runs have been produced. The case miner computes ranks, disagreement buckets, and surface-overlap proxies automatically, then emits query, positive, and top-ranked negative snippets from pre-defined disagreement buckets. The case entries in this catalog are code-selected rather than manually browsed or cherry-picked. The short explanations below are manual interpretations written after export for those emitted cases. They are not used as the statistical estimator; the aggregate counts and threshold sweeps should be read together with this catalog.

The main qualitative pattern is not that FULL-ATTN is simply worse. Full token access preserves decisive local details when exact APIs, rare identifiers, constants, return paths, or error strings are the real evidence. The same access also makes the reader sensitive to locally useful tokens that can lower NTP loss without identifying the qrel-positive document. Compact or structured exposure often acts as a useful bottleneck: it suppresses some incidental local matches and gives more credit to operation-level intent, root cause, or whole-function semantics. This explains how a model can have lower token-level loss while inducing weaker retriever supervision on some CoIR tasks, without implying that every local match is spurious.

M.1 Automatic Mining Protocol and Aggregate Counts

We form disagreement buckets from qrel-positive ranks, not from manual labels. FULL-ATTN is the baseline, and the comparison side is the best rank among EOS STATE, MEAN STATE, APPENDED LATENTS $K = 1$, APPENDED LATENTS $K = 8$, EXTRACTED LATENTS $D = 8$, and QUERY EVIDENCE top- $k = 8$. We use this oracle comparison only to mine disagreement cases; endpoint claims use individual operators. Surface-overlap flags are automatic lexical proxies over the retrieved positive and top-ranked negative, including identifier/API overlap, rare-token overlap, exact substring overlap, and near-duplicate line evidence. These flags identify local-evidence alignment, not semantic-error labels by definition. The catalog below is produced by the fixed miner before manual inspec-

Table 23: Representative automatically mined disagreement cases. This table is the compact case-summary view corresponding to the longer task-wise catalog below. Manual text interprets code-emitted cases after selection; the appendix gives the full protocol and catalog.

Case	Contrast and interpretation
AppsRetrieval q7056	<i>Structured win.</i> FULL-ATTN misses; APPENDED LATENTS $K = 8$ ranks the positive first. Full token access follows near-duplicate loop/control-flow shape, while the compact row recovers the intended algorithmic operation.
COIR Code- SearchNet q176529	<i>Structured win.</i> FULL-ATTN rank 10; APPENDED LATENTS $K = 8$ rank 1. The structured row promotes the whole-operation comment above helper-level evidence that is locally plausible but not the qrel target.
StackOverflow QA q18496	<i>Structured win.</i> FULL-ATTN misses; APPENDED LATENTS $K = 8$ and EXTRACTED LATENTS $D = 8$ rank the positive first. The positive answer concerns an AspectJ instrumentation root cause, while full access is distracted by generic exception/thread cues.
CodeSearchNet CC q175016	<i>Full-token win.</i> FULL-ATTN rank 1; APPENDED LATENTS $K = 8$ rank 10. The gold continuation depends on exact event-switch and JSON-unmarshal details; compression removes useful local evidence rather than only noise.

tion; manual explanations only interpret the code-emitted cases and do not affect which cases enter the catalog. The statistics in Tables 24–26 are also computed before manual inspection.

Table 24: Disagreement counts from the case-study miner across the completed CoIR case tasks. “Comparison” means the best rank among EOS STATE, MEAN STATE, APPENDED LATENTS $K = 1$, APPENDED LATENTS $K = 8$, EXTRACTED LATENTS $D = 8$, and QUERY EVIDENCE top- $k = 8$. Counts are query-level at top-10.

Task	Query-level top-10 counts
AppsRetrieval	Queries 3765; comparison rescue 587; comparison better 346; FULL-ATTN rescue 46; FULL-ATTN better 79; agreement failure 2295.
COIR CodeSearch-Net	Queries 8122; comparison rescue 859; comparison better 1669; FULL-ATTN rescue 85; FULL-ATTN better 279; agreement failure 1742.
CodeFeedbackMT	Queries 13277; comparison rescue 697; comparison better 1569; FULL-ATTN rescue 250; FULL-ATTN better 621; agreement failure 3594.
CodeFeedbackST	Queries 31306; comparison rescue 1299; comparison better 5169; FULL-ATTN rescue 213; FULL-ATTN better 1187; agreement failure 1946.
CodeSearchNet CC	Queries 8122; comparison rescue 831; comparison better 1711; FULL-ATTN rescue 63; FULL-ATTN better 181; agreement failure 1643.
CodeTransOcean Contest	Queries 221; comparison rescue 12; comparison better 36; FULL-ATTN rescue 0; FULL-ATTN better 7; agreement failure 19.
CodeTransOcean DL	Queries 180; comparison rescue 24; comparison better 65; FULL-ATTN rescue 0; FULL-ATTN better 4; agreement failure 11.
CosQA	Queries 500; comparison rescue 104; comparison better 127; FULL-ATTN rescue 6; FULL-ATTN better 21; agreement failure 133.
StackOverflowQA	Queries 1994; comparison rescue 135; comparison better 225; FULL-ATTN rescue 1; FULL-ATTN better 13; agreement failure 79.
SyntheticText2SQL	Queries 5851; comparison rescue 331; comparison better 1716; FULL-ATTN rescue 54; FULL-ATTN better 248; agreement failure 453.
All micro	Queries 73338; comparison rescue 4879; comparison better 12633; FULL-ATTN rescue 718; FULL-ATTN better 2640; agreement failure 11915.

Table 25: Automatic surface-overlap proxy rates at the default threshold 0.03. A row is surface-aligned when the top-ranked negative has stronger lexical-overlap evidence than the qrel-positive under the automatic proxy. Structured-favorable rows are cases where the comparison side ranks the positive higher than FULL-ATTN; FULL-ATTN-favorable rows are the reverse. The proxy supports a local-evidence-sensitivity analysis; manual case inspection determines whether the overlap is spurious or valid evidence.

Bucket	Surface-overlap proxy summary
Structured favorable	Queries 17512; FULL-ATTN surface-aligned 32.3%; comparison surface-aligned 18.2%; FULL-ATTN only 2872; comparison only 402; mean delta gap 0.0512.
FULL-ATTN favorable	Queries 3358; FULL-ATTN surface-aligned 21.5%; comparison surface-aligned 29.9%; FULL-ATTN only 145; comparison only 425; mean delta gap -0.0409.
Agreement success	Queries 40553; FULL-ATTN surface-aligned 3.8%; comparison surface-aligned 3.7%; FULL-ATTN only 84; comparison only 67; mean delta gap -0.0002.
Agreement failure	Queries 11915; FULL-ATTN surface-aligned 46.6%; comparison surface-aligned 43.7%; FULL-ATTN only 1115; comparison only 762; mean delta gap 0.0047.

M.2 Task-Wise Representative Cases

AppsRetrieval. AppsRetrieval makes local-evidence sensitivity visible because many candidate solutions share contest-style variables,

Table 26: Threshold sensitivity for the automatic surface-overlap proxy on the two directional disagreement buckets. The sign pattern is stable across thresholds: when structured exposure is favored, the FULL-ATTN side has more surface-aligned negatives; when FULL-ATTN is favored, the comparison side has more surface-aligned negatives.

Bucket		Thresh.	Counts and paired test
Structured favorable	fa-	0.01	Queries 17512; FULL-ATTN surface-aligned 39.6%; comparison surface-aligned 22.1%; paired $p < 10^{-300}$.
Structured favorable	fa-	0.03	Queries 17512; FULL-ATTN surface-aligned 32.3%; comparison surface-aligned 18.2%; paired $p < 10^{-300}$.
Structured favorable	fa-	0.05	Queries 17512; FULL-ATTN surface-aligned 27.5%; comparison surface-aligned 15.7%; paired $p < 10^{-300}$.
Structured favorable	fa-	0.10	Queries 17512; FULL-ATTN surface-aligned 21.0%; comparison surface-aligned 12.0%; paired $p = 8.1 \times 10^{-288}$.
FULL-ATTN favorable	fa-	0.01	Queries 3358; FULL-ATTN surface-aligned 26.7%; comparison surface-aligned 36.6%; paired $p = 5.6 \times 10^{-39}$.
FULL-ATTN favorable	fa-	0.03	Queries 3358; FULL-ATTN surface-aligned 21.5%; comparison surface-aligned 29.9%; paired $p = 7.3 \times 10^{-33}$.
FULL-ATTN favorable	fa-	0.05	Queries 3358; FULL-ATTN surface-aligned 18.5%; comparison surface-aligned 25.6%; paired $p = 8.0 \times 10^{-28}$.
FULL-ATTN favorable	fa-	0.10	Queries 3358; FULL-ATTN surface-aligned 14.4%; comparison surface-aligned 21.0%; paired $p = 6.2 \times 10^{-29}$.

loops, grids, and arithmetic. The useful contrast is between whole-algorithm intent and local code shape.

- q5541: FULL-ATTN misses while EOS STATE ranks the positive first; FULL-ATTN follows local arithmetic/control-flow shape, while EOS STATE recovers the intended algorithmic solution.
- q7056: FULL-ATTN misses while APPENDED LATENTS $K = 8$ ranks the positive first; the compact row is less distracted by near-duplicate loop structure.
- q7513: FULL-ATTN misses while EOS STATE ranks the positive first; the failure is consistent with over-weighting problem-format and variable-surface similarity.
- q8003: FULL-ATTN misses while APPENDED LATENTS $K = 1$ ranks the positive first; the positive requires the global transformation rather than a local coding idiom.
- q7858: FULL-ATTN misses while EXTRACTED LATENTS $D = 8$ ranks the positive first; extracted memories capture the high-level contest operation better than raw token overlap.
- q5353: FULL-ATTN rank 10 versus APPENDED LATENTS $K = 8$ rank 1; the structured row moves the gold solution above locally similar distractors.
- q5021: FULL-ATTN rank 10 versus EOS STATE rank 1; a compact endpoint state is enough to represent the intended solution and avoids local-code distractors.
- q6283: FULL-ATTN rank 10 versus APPENDED LATENTS $K = 1$ rank 1; the bottleneck favors problem-level intent over exact implementation fragments.
- q5183: FULL-ATTN rank 10 versus QUERY EVIDENCE rank 1; selected evidence tokens recover the decisive clue that full attention dilutes among many local matches.
- q8606: FULL-ATTN rank 1 while comparisons miss; this is the positive side of full exposure, where exact local code evidence identifies the solution.

1255	• q5037: FULL-ATTN rank 1 while comparisons miss; exact implementation detail is decisive and compression loses it.	1300
1256		1301
1257		1302
1258	• q5278: FULL-ATTN rank 1 while structured rows are lower; local implementation constraints are the useful evidence rather than incidental overlap.	1303
1259		
1260		
1261		
1262	• q5396: FULL-ATTN rank 1 while comparisons miss; the positive is recovered from fine-grained code structure.	
1263		
1264		
1265	• q5989: FULL-ATTN rank 1 while comparisons miss; this case shows that compression can discard exact contest-solution details.	
1266		
1267		
1268	• q5363: all systems miss; the candidate set contains many solution-shape neighbors and does not give a clean exposure-specific contrast.	
1269		
1270		
1271		
1272	• q8249: all systems miss; the gold solution is not separated well by either local details or compact summaries.	
1273		
1274		
1275	• q7724: all rows miss; the candidate set contains several highly similar contest solutions, so the error should not be attributed to exposure alone.	
1276		
1277		
1278		
1279	CosQA. CosQA is a natural-language-to-code task. FULL-ATTN often follows literal API or snippet tokens, while structured rows sometimes recover the requested operation. The boundary cases show that exact API evidence can still be useful.	
1280		
1281		
1282		
1283		
1284	• q20207: FULL-ATTN misses while EOS STATE ranks the positive first; the compact state recovers the requested code behavior rather than a local API neighbor.	
1285		
1286		
1287		
1288	• q20296: FULL-ATTN misses while QUERY EVIDENCE ranks the positive first; query-conditioned evidence selection keeps the decisive operation.	
1289		
1290		
1291		
1292	• q20114: FULL-ATTN misses while APPENDED LATENTS $K = 8$ ranks the positive first; the latent exposure suppresses a surface-near but semantically wrong implementation.	
1293		
1294		
1295		
1296	• q20415: FULL-ATTN misses while EOS STATE ranks the positive first; the query asks for operation intent, not merely matching code vocabulary.	
1297		
1298		
1299		
	• q20209: FULL-ATTN misses while EOS STATE ranks the positive first; a shorter source summary is enough to separate the intended code from token-similar distractors.	1304
		1305
		1306
		1307
	• q20283: FULL-ATTN rank 10 versus EOS STATE rank 1; FULL-ATTN keeps the positive barely in the top-10 but ranks local surface matches above it.	1308
		1309
		1310
	• q20119: FULL-ATTN rank 9 versus EOS STATE rank 1; the compact state promotes the whole-function behavior.	1311
		1312
		1313
	• q20105: FULL-ATTN rank 7 versus EOS STATE rank 1; the error pattern is a soft version of the same surface-neighbor failure.	1314
		1315
		1316
	• q20584: FULL-ATTN rank 1 while comparisons miss; the positive depends on exact code/API evidence that compression discards.	1317
		1318
		1319
	• q20241: FULL-ATTN rank 1 while comparisons miss; full token access preserves the decisive local pattern.	1320
		1321
		1322
		1323
	• q20214: FULL-ATTN rank 1 while APPENDED LATENTS $K = 8$ ranks it lower; this is a legitimate FullAttn win on a precise implementation cue.	1324
		1325
		1326
		1327
	• q20542: FULL-ATTN rank 1 while structured rows are lower; the case depends on exact implementation vocabulary rather than only semantic abstraction.	1328
		1329
		1330
		1331
	• q20350: all systems miss; the retrieved neighbors are semantically close enough that the case is better treated as benchmark ambiguity than architecture evidence.	1332
		1333
		1334
	• q20430: all systems miss; query intent and local API cues are both insufficient to separate the gold from close candidates.	1335
		1336
		1337
	• q20126: all systems miss; this is a useful guard against overclaiming any single failure mode.	1338
		1339
		1340
		1341
	• q20368: all systems retrieve close neighbors; this is a boundary case where qrel incompleteness or near-duplicate functions may dominate the apparent error.	

SyntheticText2SQL. Text2SQL is useful for separating schema/operator intent from domain-word overlap. Several FULL-ATTN failures retrieve SQL with overlapping table names or literals but the wrong operation.

- q101107: FULL-ATTN misses while APPENDED LATENTS $K = 8$ ranks the positive first; structured exposure retrieves the intended SQL operation instead of a table-name neighbor.
- q100118: FULL-ATTN misses while APPENDED LATENTS $K = 8$ ranks the positive first; the latent row follows query semantics over literal overlap.
- q101973: FULL-ATTN misses while QUERY EVIDENCE ranks the positive first; evidence selection keeps the decisive schema/operator clue.
- q104187: FULL-ATTN misses while APPENDED LATENTS $K = 8$ ranks the positive first; the failure is consistent with over-weighting domain-token overlap.
- q105048: FULL-ATTN misses while EOS STATE ranks the positive first; the compact representation better captures the required SQL transformation.
- q100123: FULL-ATTN misses while a comparison row ranks the positive first; schema words alone are not enough, and the correct operator relation matters.
- q100326: FULL-ATTN rank 10 versus APPENDED LATENTS $K = 8$ rank 1; the positive is present but buried below surface-similar distractors.
- q100865: FULL-ATTN rank 10 while a comparison row ranks the positive first; the structured row better separates aggregation or filtering intent from domain tokens.
- q101967: FULL-ATTN rank 10 versus APPENDED LATENTS $K = 8$ rank 1; structured exposure promotes the correct SQL intent.
- q102502: FULL-ATTN rank 10 versus APPENDED LATENTS $K = 8$ rank 1; the comparison row is less tied to overlapping literals.

- q100778: FULL-ATTN rank 1 while comparisons miss; exact schema and literal evidence are decisive.
- q100221: FULL-ATTN rank 1 while comparisons miss; compression loses a precise SQL detail.
- q100645: FULL-ATTN rank 1 while EOS STATE ranks it lower; full access preserves the exact query form.
- q100894: FULL-ATTN rank 1 while comparisons are lower; local schema details matter more than abstraction.
- q103901: FULL-ATTN rank 1 while comparisons are lower; the case shows that filtering surface overlap can also remove needed evidence.
- q101132: FULL-ATTN rank 1 while comparisons are lower; the SQL answer is identified by precise column/table constraints.
- q102769: FULL-ATTN rank 1 while comparisons are lower; literal schema evidence is the correct retrieval cue.
- q103040: FULL-ATTN rank 1 while comparisons are lower; the row illustrates a FullAttn strength on exact SQL form.
- q105539: FULL-ATTN rank 1 while comparisons are lower; compression loses a useful local SQL detail.
- q104235: all rows miss; the gold SQL is hard to identify from the available local evidence, so it is not a clean exposure-specific failure.
- q102183: all rows miss; the negative set contains query-near SQL variants that no exposure pattern cleanly separates.
- q102430: all rows miss; this is another boundary case where the case study should not pretend the mechanism is sufficient.

COIRCodeSearchNetRetrieval. In code-to-comment retrieval, FULL-ATTN often overweights callees, helper names, or exact identifiers. Structured exposure more often retrieves the comment that summarizes the whole function. Exact function and signature matches remain a real strength of FULL-ATTN.

1431	• q176660: FULL-ATTN misses while EOS	• q177439: FULL-ATTN rank 1 while com-	1477
1432	STATE ranks the positive first; FULL-ATTN	parisons miss; ListObject is an exact	1478
1433	retrieves a NodeIsEmpty helper comment	API/signature match.	1479
1434	rather than the outer leave-cluster precondition.		
1435			
1436	• q177820: FULL-ATTN misses while EOS	• q180274: FULL-ATTN rank 1 while com-	1480
1437	STATE ranks the positive first; FULL-ATTN	parisons miss; ErrorService is a precise	1481
1438	follows PointerOf, while EOS STATE re-	option setter.	1482
1439	trieves the visited-map operation.		
1440			
1441	• q182409: FULL-ATTN misses while EOS	• q181361: FULL-ATTN rank 1 while compar-	1483
1442	STATE ranks the positive first; FULL-ATTN	isons miss; benchmark-run naming and signa-	1484
1443	follows LoadPivnetRelease, while	ture evidence make the target comment easy	1485
1444	EOS STATE retrieves the constructor-level	for full token access.	1486
1445	Differ comment.		
1446			
1447	• q177004: FULL-ATTN misses while AP-	• q176330: FULL-ATTN rank 1 while AP-	1487
1448	PENDED LATENTS $K = 1$ ranks the positive	PENDED LATENTS $K = 8$ ranks it lower;	1488
1449	first; the structured row captures the Ceph vol-	the stop-channel interrupt helper needs local	1489
1450	ume replication intent.	signal-handling evidence.	1490
1451			
1452	• q180949: FULL-ATTN misses while EX-	• q175647: FULL-ATTN rank 1 while	1491
1453	TRACTED LATENTS $D = 8$ ranks the positive	QUERYEVIDENCE ranks it lower;	1492
1454	first; FULL-ATTN is attracted to logging im-	DialAndSend directly maps to open-	1493
1455	plementation detail rather than logging-level	ing an SMTP connection, sending email, and	1494
1456	configuration.	closing it.	1495
1457			
1458	• q178245: FULL-ATTN rank 10 versus EOS	• q176987: both FULL-ATTN and EOS	1496
1459	STATE rank 1; FULL-ATTN retrieves a relay-	STATE place the positive at rank 4; the top	1497
1460	timer pool helper and misses the target release	qrel-negative is a close storage-pool add oper-	1498
1461	operation.	ation, suggesting qrel or alias ambiguity.	1499
1462			
1463	• q176529: FULL-ATTN rank 10 versus AP-	• q177039: both systems place the positive	1500
1464	PENDED LATENTS $K = 8$ rank 1; the struc-	near the top but not first; the top result is a	1501
1465	tured model ranks the whole-operation com-	strong rsyncSend semantic neighbor and	1502
1466	ment before helper-level evidence.	may reflect incomplete qrels.	1503
1467			
1468	• q174802: FULL-ATTN rank 10 versus AP-	• q175017: all systems miss; the gold com-	1504
1469	PENDED LATENTS $K = 1$ rank 1; group-	ment is short and underdescriptive, so expo-	1505
1470	membership checking is captured by struc-	sure choice is not the only bottleneck.	1506
1471	tured exposure, while FULL-ATTN drifts to a		
1472	watcher function.		
1473			
1474	• q174804: FULL-ATTN rank 10 versus EOS	CodeSearchNetCCRetrieval. This task contains	1507
1475	STATE rank 1; EOS STATE retrieves the	many near-neighbor code fragments from the same	1508
1476	watcher update-choice abstraction.	package or API family. It gives many examples	1509
		where FULL-ATTN ranks adjacent setters, wrap-	1510
		pers, or callers above the exact continuation, and	1511
		also many examples where exact local API order is	1512
		the correct evidence.	1513
	• q174879: FULL-ATTN misses while AP-	• q174879: FULL-ATTN misses while AP-	1514
	PENDED LATENTS $K = 8$ ranks the	PENDED LATENTS $K = 8$ ranks the	1515
	positive first; FULL-ATTN locks onto an-	positive first; FULL-ATTN locks onto an-	1516
	other SetEndpoints caller rather than the	other SetEndpoints caller rather than the	1517
	resolver-address update.	resolver-address update.	1518
	• q175978: FULL-ATTN misses while EOS	• q175978: FULL-ATTN misses while EOS	1519
	STATE ranks the positive first; FULL-ATTN	STATE ranks the positive first; FULL-ATTN	1520
	follows a different ServerError action	follows a different ServerError action	1521

1522	path, while EOS STATE retrieves the HTTP	• q177838: FULL-ATTN keeps the positive at	1568
1523	500 formatter.	rank 2 while comparisons miss; reflection ad-	1569
1524	• q177840: FULL-ATTN misses while AP-	dressability depends on local CanAddr, New,	1570
1525	PENDED LATENTS $K = 1$ ranks the positive	Elem, and Set API order.	1571
1526	first; the target is a short <code>Field.String()</code>	• q180569: FULL-ATTN keeps the positive at	1572
1527	rendering body, but FULL-ATTN retrieves a	rank 3 while comparisons miss; the basic-auth	1573
1528	line/string parser.	credential map and user/password comparison	1574
1529	• q178598: FULL-ATTN misses while AP-	are local details.	1575
1530	PENDED LATENTS $K = 1$ ranks the posi-	• q175589: FULL-ATTN keeps the positive	1576
1531	tive first; FULL-ATTN chooses router handler-	at rank 3 while comparisons miss; the target	1577
1532	composition surface, while the positive is the	combines partial names, content type, exten-	1578
1533	closure that iterates over handlers.	sion fixing, and boxed template lookup.	1579
1534	• q179741: FULL-ATTN misses while EOS	• q175016: FULL-ATTN rank 1 versus AP-	1580
1535	STATE ranks the positive first; FULL-ATTN	PENDED LATENTS $K = 8$ rank 10; watcher	1581
1536	ranks adjacent <code>DeleteCookiesParams</code>	<code>Next()</code> requires detailed event-switch and	1582
1537	setters, while EOS STATE retrieves the <code>Do</code>	JSON-unmarshal context.	1583
1538	method that executes the command.	• q179342: FULL-ATTN rank 1 versus MEAN	1584
1539	• q177851: FULL-ATTN rank 10 versus EOS	STATE rank 9; <code>AppEngine blob Stat</code> de-	1585
1540	STATE rank 1; FULL-ATTN follows an Open-	pends on exact namespace/datastore lookup	1586
1541	Tracing helper with <code>ContextWithSpan</code> ,	and mismatch handling.	1587
1542	while EOS STATE retrieves the direct	• q180837: FULL-ATTN rank 1	1588
1543	<code>context.WithValue</code> implementation.	versus <code>QUERYEVIDENCE</code> rank	1589
1544	• q177089: FULL-ATTN rank 10 versus EOS	7; the <code>tiny</code> method is simply	1590
1545	STATE rank 1; FULL-ATTN is attracted to	<code>a.Auth.CanAuthenticate(a.Host)</code> ,	1591
1546	nearby container-backup SQL joins, while	where abstraction can over-generalize.	1592
1547	EOS STATE recovers expiry timestamp un-	• q181799: FULL-ATTN rank 1 versus	1593
1548	marshalling.	<code>QUERYEVIDENCE</code> rank 4; signed-URL con-	1594
1549	• q178377: FULL-ATTN rank 10 versus EX-	struction depends on literal Hawk request-	1595
1550	TRACTED LATENTS $D = 8$ rank 1; EX-	auth fields.	1596
1551	TRACTED LATENTS captures the one-shot	• q174617: both systems rank the positive sec-	1597
1552	CAS error broadcast guard.	ond behind another STM serializable wrapper,	1598
1553	• q178493: FULL-ATTN rank 10 versus AP-	a near-duplicate boundary case.	1599
1554	PENDED LATENTS $K = 1$ rank 1; the latent	• q175099: both systems rank the positive sec-	1600
1555	row keeps the upward directory search and	ond behind an <code>etcd</code> defragmentation helper	1601
1556	stopping condition.	from the same API family.	1602
1557	• q175879: FULL-ATTN rank 10 versus EOS	• q175116: both systems rank the positive sec-	1603
1558	STATE rank 1; FULL-ATTN follows path	ond behind a random-string generator neigh-	1604
1559	traversal tokens, while EOS STATE selects	bor, while the gold is the unique-string dedu-	1605
1560	set union over parent and child string lists.	plication wrapper.	1606
1561	• q175087: FULL-ATTN rank 1 while com-	• q175277: both systems rank the positive sec-	1607
1562	parisons miss; exact <code>url.Values</code> lookup	ond behind a merge-iterator constructor that	1608
1563	and <code>strconv.ParseBool</code> conversion are	calls <code>initHeap</code> , suggesting qrel incomplete-	1609
1564	decisive.	ness.	1610
1565	• q177285: FULL-ATTN rank 1 while com-		
1566	parisons miss; the SQL delete helper requires		
1567	exact <code>RowsAffected</code> post-checks.		

1611	• q176195: both systems rank a nearby job-	• q18654: FULL-ATTN misses while EOS	1656
1612	agent constructor before the <code>Spyglass</code> con-	STATE ranks the positive first; check-	1657
1613	structor, a package-level constructor ambigu-	box/onChange code surface hides the simple	1658
1614	ity.	root cause that JavaScript was disabled.	1659
1615	• q174737: all systems miss; the constructor	• q18726: FULL-ATTN rank 10 versus EOS	1660
1616	immediately calling <code>StringValue.Set</code>	STATE rank 1; React warning stack traces in-	1661
1617	is a local-neighbor trap for every exposure.	introduce surface noise, while structured rows	1662
1618	• q174916: all systems miss; the gold wraps a	retrieve the Babel plugin/component-stack an-	1663
1619	lease with a prefix, while systems prefer <code>etcd</code>	swer.	1664
1620	connection construction with <code>NewLease</code> .	• q18362: FULL-ATTN rank 9 versus AP-	1665
1621	• q174971: all systems miss; the cor-	PENDED LATENTS $K = 1$ rank 1; Android	1666
1622	pus contains wrapper calls around	service/activity surface hides the lifecycle root	1667
1623	<code>IsDirWriteable</code> , making exact im-	cause.	1668
1624	plementation retrieval hard for all rows.	• q18226: FULL-ATTN rank 9 versus	1669
1625	• q175020: all systems miss; dense v2 er-	QUERYEVIDENCE rank 1; QUERYEVIDENCE	1670
1626	ror branches compete with the target node	captures the Camel aggregation release-	1671
1627	<code>Read()</code> value/error case.	strategy answer.	1672
1628	StackOverflowQA. StackOverflowQA empha-	• q18640: FULL-ATTN rank 10 versus EOS	1673
1629	sizes root-cause retrieval from long questions.	STATE rank 1; the gold one-shot git editor	1674
1630	FULL-ATTN often follows code snippets, stack	override is <code>EDITOR=nano git commit</code>	1675
1631	traces, error strings, and configuration blocks,	-a, while FULL-ATTN retrieves a Sublime	1676
1632	while structured exposure more often recovers the	command-line issue.	1677
1633	short root-cause answer. FULL-ATTN still wins	• q19220: FULL-ATTN rank 1 while AP-	1678
1634	when the answer depends on exact API order or	PENDED LATENTS $K = 1$ ranks it lower;	1679
1635	language-specific convention.	Firestore persistence requires a precise lifecy-	1680
1636	• q18179: FULL-ATTN misses while AP-	cle/API ordering constraint.	1681
1637	PENDED LATENTS $K = 1$ ranks the	• q19140: FULL-ATTN rank 1 while EOS	1682
1638	positive first; FULL-ATTN retrieves <code>nginx</code>	STATE ranks it lower; Kotlin <code>Parcelable</code> in-	1683
1639	rewrite/config neighbors, while the gold an-	terop requires the exact <code>@JvmField</code> conven-	1684
1640	swer is <code>php-fpm</code> process exhaustion.	tion.	1685
1641	• q18249: FULL-ATTN misses while EOS	• q19241: FULL-ATTN rank 1 while MEAN	1686
1642	STATE ranks the positive first; FULL-ATTN	STATE ranks it lower; the datepicker solution	1687
1643	follows C-like <code>fseek/read/sizeof</code> sur-	depends on exact plugin parameters such as	1688
1644	face, while EOS STATE retrieves the D file-	<code>startDate</code> and <code>changeDate</code> .	1689
1645	read solution.	• q18864: FULL-ATTN rank 1 while EOS	1690
1646	• q18496: FULL-ATTN misses while AP-	STATE ranks it lower; FULL-ATTN distin-	1691
1647	PENDED LATENTS $K = 8$ and EX-	guishes top words per LDA topic from neigh-	1692
1648	TRACTED LATENTS $D = 8$ rank the posi-	boring LDA subtasks.	1693
1649	tive first; the answer concerns <code>AspectJ</code> han-	• q19698: FULL-ATTN rank 1 while MEAN	1694
1650	dler/instrumentation rather than generic ex-	STATE ranks it lower; exact Eclipse debugger	1695
1651	ception/thread cues.	step-into/step-over wording is decisive.	1696
1652	• q19318: FULL-ATTN misses while several	• q18955: all systems miss; Angular, Docker,	1697
1653	structured rows rank the positive first; the	<code>nginx</code> , and content-length cues hide a	1698
1654	root cause is <code>macOS/Xcode</code> command-line	troubleshooting answer about reinstalling	1699
1655	tools/MacPorts, not Python egg permissions.	<code>node_modules</code> .	1700

- q18246: all systems miss; Maven/Tomcat/IntelliJ/Jersey cues dominate, but the answer is browser cache.
- q18694: all systems miss; JSX map/style surface leads systems away from the ternary conditional rendering answer.
- q19391: all systems miss; Android test-code and UI-thread cues produce dispersed retrieval, and the gold requires stacktrace diagnosis.

The cases support an exposure-tradeoff explanation. FULL-ATTN failures are often not arbitrary: the top negatives are locally plausible documents that share identifiers, APIs, error strings, SQL schema words, or problem-format tokens. Structured exposure does not merely remove information; in many examples it changes the unit of evidence from local token match to operation intent or root cause. Conversely, FULL-ATTN wins when exact local evidence is required. Full access can lower reader loss by exploiting local details, but only some of those details provide useful document-level retriever supervision.